

---

# Maximally Robust Satisficing Bayesian Optimization

---

Samuli Kinnunen<sup>1</sup>

Petrus Mikkola<sup>1</sup>

Antti Niskanen<sup>2</sup>

Arto Klami<sup>1</sup>

<sup>1</sup>Department of Computer Science, University of Helsinki, Finland

<sup>2</sup>ASM International N.V.

## Abstract

Many design tasks can be cast as black-box function optimization, enabling use of Bayesian optimization to find an ideal design with minimal number of trials. However, often we do not actually need the optimum but instead a sufficiently good solution is enough, for instance a material that is durable enough for its intended use. In most cases there are multiple satisfactory solutions, forming a superlevel set of the function, raising a key question of which one to prefer. We answer this by explaining why robustness to input perturbations that may occur when the solution is deployed is a good criterion and by introducing a Bayesian optimization method that efficiently finds satisficing solutions that are robust to maximally large perturbations. In contrast to previous works, we assume the inputs can be accurately controlled during optimization, but will be perturbed after the deployment.

## 1 INTRODUCTION

Bayesian optimization (BO) is a sample-efficient framework for optimizing expensive black-box functions, applied in diverse domains such as hyperparameter tuning Cho et al. [2020], materials discovery Chitturi et al. [2024], chemical synthesis Shields et al. [2021], and robotic control Lechuz-Sierra et al. [2024]. For the standard problem of finding the global optimum,  $x^* = \arg \max_x f(x)$ , there are highly polished tools. However, for many design tasks where the evaluations are extremely costly, e.g. due to involving production and testing of a new material, even a sample-efficient search may still be difficult in practice.

Often we do not, however, need the global optimum  $x^*$ , but already a sufficiently good solution that meets a given minimum quality,  $f(x^*) \geq t$ , is enough. For instance, a subcontractor may be asked to produce a component that

achieves a certain level of durability, reducing the difficulty of the problem. We call such a solution *satisficing*, following the nomenclature introduced by Herbert Simon [Simon, 1955] and subsequently used e.g. in operations research and decision-making [Reverdy et al., 2016, Long et al., 2023, Saday et al., 2025].

The set of all satisficing solutions forms a superlevel set  $S_t = \{x : f(x) \geq t\}$  of the function. In principle any solution within  $S_t$  would work, but outside of idealized conditions they are not alike. One would intuitively steer away from solutions close to the threshold of  $f(x) = t$  and prefer a solution with a higher value, but we need a more formal criterion. We argue that robustness to input noise or variation is a good one: A solution is good if  $f(x^*)$  remains satisficing when  $x^*$  is perturbed.

We draw motivation from applications where the inputs can be accurately controlled in the design (optimization) phase but are unreliable after deployment. As a concrete example, consider a cake recipe designed by professionals that measure the ingredients accurately and use consistent kitchen equipment. When commoners bake the cake following the recipe, they will not measure their tablespoons exactly and their ovens do not maintain the temperature perfectly. What ultimately matters is how good *their* cake tastes, and hence a recipe robust to this kind of deviations is a better one. The same principle is seen in many scientific applications: manual procedures and environmental factors like humidity introduce variability in laboratory chemistry [Aldeghi et al., 2021], sim-to-real transfer [Doersch and Zisserman, 2019] is a whole subfield studying specifically how to train reliable real-world models using clean simulation environments, and in design and manufacturing [Daulton et al., 2022] fabrication tolerances constraint the realizability of designs. Often the solution that is optimal under noiseless inputs can perform poorly or even lead to unsafe behavior under perturbations Oliveira et al. [2019].

To address this need, we introduce a new problem: Given a threshold  $t$  that determines the minimum acceptable value

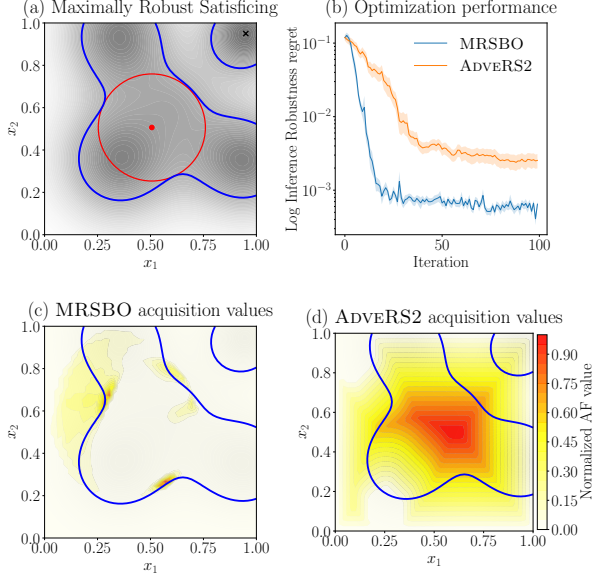


Figure 1: **(a)** The maximally robust satisficing solution (red) is a point that can be modified the most while still staying within the superlevel set defined by the threshold that needs to be satisfied (blue). Note that the global maximum (black cross) can be arbitrarily far. **(b)** Our MRSBO method drastically outperforms the best previous method (AdVERS2) adapted for our setup; it also optimizes for the robustness radius, but assumes input perturbations already during optimization. **(c-d)** Acquisition functions for the two methods during late stages of optimization. MRSBO explores the parts of the level set that are informative of the robustness radius, whereas AdVERS2 wastes evaluations for learning the target within the superlevel set.

for the function, find a solution  $x^*$  that is robust to *maximally large* input deviations. During optimization we assume  $f(x)$  can be evaluated for arbitrary  $x$ , but the solution is eventually deployed in an environment where the inputs are perturbed by an unknown noise. That is, instead of  $f(x^*)$  we observe  $f(x^* + \delta)$ . The task is illustrated in Figure 1, showing also how our method maximizing the information gain about the maximal robustness radius solves the problem efficiently by primarily evaluating the function around selected parts of the level set, not wasting evaluations to learn the function accurately within the superlevel set.

While our problem setup is new, it is closely related to the concept of *Robust Bayesian Optimization (RBO)*, generally referring to optimization methods finding solutions that remain effective under input perturbations. Previous RBO methods can be broadly categorized into two types. *Expectation Robustness* [Yang et al., 2023, Fröhlich et al., 2020] optimizes expected performance under a known perturbation distribution  $p(\delta)$ ,  $x^* = \arg \max_x E_{\delta \sim p(\delta)} [f(x + \delta)]$ , with *Distributionally Robust Optimization* [Kirschner et al.,

2020, Shapiro et al., 2023] accounting also for uncertainty in the perturbation distribution. These methods ensure the expected performance to be good, but there are *no* guarantees for the solution to satisfy  $f(x) > t$  for any  $x$ .

*Worst-case Robustness* [Bertsimas et al., 2010, Bogunovic et al., 2018] is closer to our setup, searching for a solution that performs optimally under the most adverse perturbation as  $x^* = \arg \max_x \min_{\delta} f(x + \delta)$ . Most methods in this family, like StableOpt by Bogunovic et al. [2018], directly optimize for the worst-scared performance, but recently Saday et al. [2025] connected the worst-case robustness to satisficing solutions. They also maximize the magnitude of the perturbations that can be allowed while ensuring the solution remains above the threshold  $t$ . However, there is a fundamental difference in the learning setup: Similar to other worst-case robustness methods, Saday et al. [2025] assume already the optimization is done in an adversarial environment. This is in stark contrast to our setup where the inputs are clean during optimization. Their method can be used to solve our problem by artificially inducing perturbations during optimization, but this requires assuming a maximum level of perturbation. Figure 1 shows how this approach in practice fails even when given oracle information about the maximum perturbation, with substantially worse convergence, poor final regret, and wasteful evaluations.

**Contributions.** Our main contributions are: (1) We formalize Maximally Robust Satisficing Bayesian Optimization problem setup for learning solutions that remain sufficiently good under maximal input perturbation. (2) We provide practical algorithm MRSBO for solving it, based on the information gain about the maximally robust solution with an efficient construction for the acquisition function. (3) We demonstrate that MRSBO vastly outperforms robust and non-robust alternatives on a variety of synthetic and real-world robust optimization problems.

## 2 BACKGROUND

### 2.1 BAYESIAN OPTIMIZATION FRAMEWORK

Bayesian Optimization, in its standard form, is a global optimization method for solving

$$x^* = \arg \max_{x \in \mathcal{X}} f(x), \quad (1)$$

where  $\mathcal{X} \subseteq R^d$  is a  $d$ -dimensional design space and  $f : \mathcal{X} \rightarrow R$  is an unknown function that is expensive to evaluate. To optimize the function, we can only evaluate inputs  $x$  to produce an evaluation  $y = f(x)$ . BO algorithm learns cheap surrogate model, typically a Gaussian process, that approximates the real function and selects the next evaluation by maximizing an acquisition function to identify the evaluation that maximally benefits the optimization process. **Surrogate model** approximates the true objective

function  $f$ . A Gaussian process (GP) Williams and Rasmussen [1995] is the most commonly used surrogate model, providing a distribution over function values  $f(x)$  characterized by a mean function  $\mu(x)$  and a covariance (kernel) function  $k(x, x')$ . Given observations  $D_t = \{(x_i, y_i)\}_{i=1}^t$ , the posterior predictive distribution at a test point  $x$  is Gaussian with mean  $\mu_t(x) = k_t(x)^\top (K + \sigma_\epsilon^2 I)^{-1} y$  and variance  $\sigma_t^2(x) = k(x, x) - k_t(x)^\top (K + \sigma_\epsilon^2 I)^{-1} k_t(x)$ , where  $k_t(x) = [k(x_i, x)]_{i=1}^t$ ,  $K = [k(x_i, x_j)]_{i,j=1}^t$  is the kernel matrix of the training inputs,  $y = [y_1, \dots, y_t]^\top$ , and  $\sigma_\epsilon^2$  denotes the observation noise variance.

**Acquisition function (AF)**  $\alpha(x)$  scores the value of evaluating an input point  $x$ . At each Bayesian optimization iteration, the next query point is selected by solving  $x_{t+1} = \arg \max_{x \in \mathcal{X}} \alpha(x)$ . Popular single-objective acquisition functions include Upper Confidence Bound (UCB), Expected Improvement (EI) Jones et al. [1998] and Max-value Entropy Search (MES) Wang and Jegelka [2017].

### 3 PROBLEM DEFINITION

We consider design tasks where a solution  $x^*$  is determined in controlled settings, but after fixing the design it is deployed in an environment where every input  $x \in \mathcal{X}$  may experience an unknown perturbation  $\delta$  whose magnitude is bounded by an unknown robustness radius  $r > 0$ .

For any  $r$ , the admissible perturbations are collected in

$$B_r = \{\delta \in R^d : \|\delta\| < r\},$$

and we define as satisficing the  $x \in \mathcal{X}$  that remain satisfactory under all admissible perturbations: the worst-case function value stays above a given threshold  $t \in R$  as

$$\inf_{\delta \in B_r} f(x + \delta) \geq t.$$

To find the maximally robust solution, we define the *objective function*  $\rho(x)$  as the maximum robustness radius  $r$  such that all perturbed inputs yield satisficing values:

$$\rho(x) = \sup \left\{ r > 0 : \inf_{\delta \in B_r} f(x + \delta) \geq t \text{ and } x + \delta \in \mathcal{X} \right\}. \quad (2)$$

This immediately gives the final learning task that provides both the optimal solution  $x^*$  and the corresponding robustness radius  $r^*$  as

$$x^* = \arg \max_{x \in \mathcal{X}} \rho(x), \quad (3)$$

$$r^* = \rho(x^*). \quad (4)$$

Our *objective* is identical to the one recently proposed by Saday et al. [2025], but the *learning setup* is different. We assume exact evaluations  $f(x)$  during optimization, matching

---

#### Algorithm 1: Maximally Robust Satisficing Bayesian Optimization (MRSBO)

---

**Input:** Design space  $\mathcal{X}$ , GP prior  $(\mu(x), k(x, x'))$ , acquisition function  $\alpha(x)$ , budget  $T$

**Output:** Recommended point  $\tilde{x}^*$

---

```

1 Initialize data  $\mathcal{D}_0$ 
2 for  $t = 1, \dots, T$  do
3    $\{(x^{*,(s)}, r^{*,(s)})\}_{s=1}^n \leftarrow \text{SAMPLECONTINUOUS}$ 
4    $x_t \leftarrow \arg \max_{x' \in \mathcal{X}} \alpha(x', \{(x^{*,(s)}, r^{*,(s)})\}_{s=1}^n, \mathcal{D}_{t-1})$ 
5    $y_t \leftarrow f(x_t) + \epsilon_t$ 
6    $\mathcal{D}_t \leftarrow \mathcal{D}_{t-1} \cup \{(x_t, y_t)\}$ 
7 end
8  $\tilde{x}^* \leftarrow \arg \max_{x \in \mathcal{X}} \tilde{\rho}(x)$ 

```

---

the motivational example cases, whereas they assume the inputs are perturbed also during optimization (and that the perturbed inputs can, perhaps unrealistically, be observed for learning the proxy).

### 4 METHOD

This section introduces the MRSBO method for solving the above problem, summarized in Algorithm 1. It follows a standard Bayesian optimization protocol: At each iteration, candidate maximizers are sampled (Line 3), and the acquisition function is optimized to select the next evaluation point (Line 4). The proxy is updated with the new observation, and we continue until exhausting an evaluation budget  $T$ .

We here design an information-theoretic acquisition function and explain how it can be computed efficiently. The acquisition maximization step can be carried out using standard gradient-based optimization methods, e.g., as discussed by Wilson et al. [2018]; we leave description of this standard procedure in Appendix B.1.

#### 4.1 INFORMATION GAIN ABOUT $(x^*, r^*)$

We propose an acquisition strategy that maximizes the information gain about the maximum robustness radius  $r^*$  and its corresponding robust maximizer  $x^*$ . Following the symmetric mutual information formulation [Hernández-Lobato et al., 2014, Wang and Jegelka, 2017], this approach avoids direct entropy computation of the maximizer and robustness radius, which are difficult quantities even to sample.

Formally, the acquisition function for candidate  $x$  is

$$\begin{aligned}
\alpha(x) &= I(\{y, x\}; \{x^*, r^*\}) \\
&= I(\{x^*, r^*\}; \{y, x\}) \\
&= H(p(y \mid x, D_t)) \\
&\quad - E_{p(x^*, r^* \mid D_t)}[H(p(y \mid x, D_t, x^*, r^*))]
\end{aligned} \quad (5)$$

The first term of Eq. 5,  $H(p(y | x, D_t))$ , is the entropy of a Gaussian random variable and admits the analytic form

$$H(p(y | x, D_t)) = \frac{1}{2} \log(2\pi e \sigma_t^2), \quad (6)$$

where  $\sigma_t^2$  denotes the posterior predictive variance at  $x$ .

The expectation in the second term of Eq. 5 is taken with respect to the joint posterior of the robust maximizer and robustness radius, and is estimated using Monte Carlo sampling. We next discuss the approximation of the conditional entropy,  $H(p(y | x, D_t, x^*, r^*))$ , starting with a noiseless case for communicating the core idea even though in practice we always assume noisy observations.

**Noiseless conditional entropy.** In the absence of observation noise ( $\sigma_\epsilon^2 = 0$ ), the computation can be separated into two cases based on the distance  $d(x, x^*)$ . If the candidate  $x$  is within the robustness radius ( $d(x, x^*) \leq r^*$ ) then, by definition of robustness, we are guaranteed to have  $f(x) \geq t$ . When  $d(x, x^*) > r^*$ , we do not have any information about the values of  $f(x)$  unless making additional assumptions about the function.  $f(x)$  can still be arbitrarily high since the superlevel set can take any form. Hence, the conditional entropy can be expressed as:

$$H(p(y | x, D_t, x^*, r^*)) \approx \begin{cases} \log(\sqrt{2\pi e} \sigma Z) + \frac{\alpha \varphi(\alpha)}{2Z}, & \text{if } d(x, x^*) \leq r^*, \\ \frac{1}{2} \log(2\pi e \sigma_t^2), & \text{otherwise,} \end{cases} \quad (7)$$

where  $\alpha = (a - \mu)/\sigma_t$  is the standardized lower truncation point, with  $a$  being the lower bound.  $Z$  and  $\varphi$  are defined as

$$Z = 1 - \Phi(\alpha) \quad \varphi(\xi) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{1}{2}\xi^2\right),$$

where  $Z(x) = 1 - \Phi(\alpha)$  can be computed using the standard normal cumulative distribution function  $\Phi$ .

For smooth functions we could attempt lower-bounding the function also outside of the robustness radius: For  $x$  just outside the robustness radius, we necessarily have  $f(x)$  close to  $t$ . Exploiting this in practice, however, would require additional assumptions about the smoothness of the function, which are technically challenging to specify correctly [Lederer et al., 2019] and yield little empirical benefit, as shown in Appendix A.3. We therefore recommend using the above conditional entropy approximation.

**Noisy conditional entropy.** With observation noise, the guarantee that  $f(x) > t$  for  $d(x, x^*) \leq r^*$  no longer holds, but the distance still provides information. For computing the entropies, we adapt the approach of Takeno et al. [2020] proposed for noisy truncation in multi-fidelity BO context. The conditional predictive density can be expressed as

$$p(y | x, D_t, f > t) = Z(y) p(y | x, D_t), \quad (8)$$

where  $p(y | x, D_t)$  is the posterior predictive density and

$$Z(y) = \frac{p(f > t | y, x, D_t)}{p(f > t | x, D_t)}.$$

The denominator corresponds to a Gaussian tail probability, while the numerator is computed from the conditional distribution  $f | y, x, D_t \sim \mathcal{N}(u, s^2)$ , with

$$u = \mu + \frac{\sigma_f^2}{\sigma_f^2 + \sigma_\epsilon^2}(y - \mu), \quad s^2 = \sigma_f^2 - \frac{\sigma_f^4}{\sigma_f^2 + \sigma_\epsilon^2}.$$

Finally, the conditional entropy  $H(p(y | x, D_t, x^*, r^*))$  is approximated via Monte Carlo samples  $\{y_i\}_{i=1}^N$  as

$$-\frac{1}{N} \sum_{i=1}^N \log p(y_i | x, D_t, f > t) \text{ if } d(x, x^*) \leq r^*, \\ \frac{1}{2} \log(2\pi e \sigma_t^2), \quad \text{otherwise.}$$

## 4.2 SAMPLING FROM $p(x^*, r^* | D_t)$

Evaluation of the acquisition function requires computing expectations with respect to the posterior  $p(x^*, r^* | \mathcal{D}_t)$ . Since this distribution is generally intractable, we approximate the expectation using Monte Carlo integration. This reduces the problem to efficiently sampling pairs  $(x^*, r^*)$ .

Following Hernández-Lobato et al. [2014, 2016], we approximate samples from the GP posterior using Random Fourier Features (RFF). This yields differentiable sample paths for the sampling procedure in Algorithm 2.

**RFF representation** By the Bochner’s theorem, the spectral density of the GP kernel  $k$  exists and we can approximate  $k$  as an empirical expectation of the feature inner product  $k(x, x') \approx \phi^T \phi$ . For the radial basis function (RBF) kernel with the length scales  $\ell$  and the output scale  $\sigma_f$ ,  $\phi$  is an  $m$ -dimensional random feature function defined as

$$\phi(x) = \sqrt{\frac{2\sigma_f}{m}} \cos(Wx + b),$$

where the rows of  $W \in \mathbb{R}^{m \times d}$  are sampled independently as  $w \sim \mathcal{N}(0, I/\ell)$  and the phases are sampled as  $b \sim \mathcal{U}(0, 2\pi)$ . We approximate the GP prior using a Bayesian linear model

$$\tilde{f}(x) = \phi(x)^\top \theta,$$

where  $\theta \sim \mathcal{N}(0, I)$  is a Gaussian-distributed weight vector. The posterior of  $\theta | D_t$  is Gaussian,  $\theta \sim \mathcal{N}(A^{-1} \Phi^\top y, \sigma^2 A^{-1})$ , where  $\Phi = [\phi(x_1), \dots, \phi(x_t)]^\top$  is the feature matrix and  $A = \Phi^\top \Phi + \sigma^2 I$ .

**Robust maxima** To generate a single approximate posterior sample  $f_s$ , we draw both the random feature parameters  $(w_s, b_s)$  and the corresponding posterior weight vector  $\theta_s$

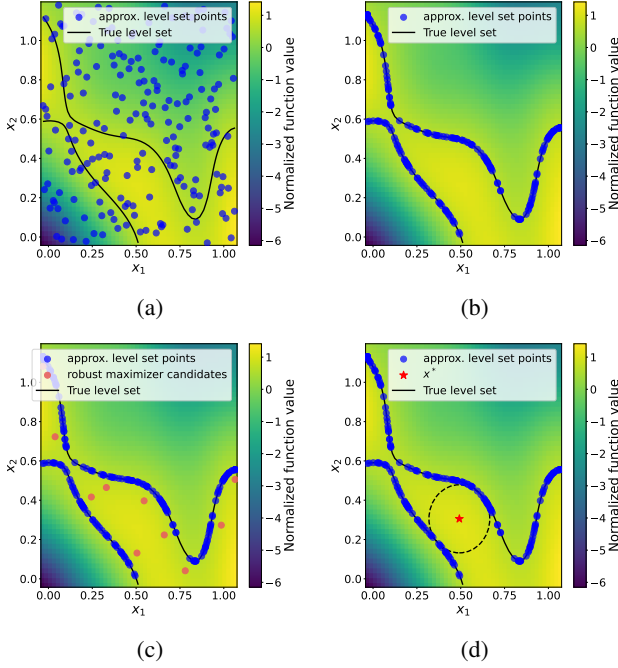


Figure 2: Sampling of  $\{(x_s^*, r_s^*)\}_{s=1}^S$ . For each  $s$ , space-filling initial candidates (a) are refined using Gauss-Newton to find the level set (b). A robust maximizer (d) is found by gradient-based optimization of LogSumExp distance to the level set samples, multi-started from a diverse subset of the level set samples pushed to the superlevel set (c).

according to the generative process above. This yields a differentiable sample function

$$\tilde{f}_s(x) = \Phi_s(x)^\top \theta_s.$$

This enables efficient Algorithm 2 for finding the robust maximizer  $(x_s^*, r_s^*)$  for each  $\tilde{f}_s$ . It is visually illustrated in Figure 2, consisting of four key steps explained next.

First, we construct an estimate of the (inflated) level set

$$\mathcal{C}_s = \{x \in \mathcal{X} : |f_s(x) - t| < \tau\},$$

for a tolerance  $\tau$ . The set  $\mathcal{C}_s$  is approximated by first sampling  $k$  space filling samples  $(x_1, \dots, x_k)$  (Fig. 2 (a)), and using the Gauss-Newton algorithm starting from each of these initial samples to minimize the squared residual  $(f_s(x_i) - t)^2$  until the tolerance is met; see Algorithm 14 in Appendix. This gives  $k$  samples covering the level set (Fig. 2 (b)).

The robust maximizer is then obtained by solving

$$x_s^* = \arg \max_{x \in \mathcal{S}_s} \left[ -\frac{1}{\beta} \log \sum_{c \in \mathcal{C}_s} \exp(-\beta d(x, c)) \right], \quad (9)$$

where the differentiable objective corresponds to a soft-min approximation of the distance to the level set. We first

---

#### Algorithm 2: SAMPLECONTINUOUS

---

**Input:** Surrogate GP model  $\mathcal{GP}$ , number of level set samples  $S$ , number of basis functions  $m$ , tolerance  $\tau$ , threshold  $t$

**Output:**  $\{(x_s^*, r_s^*)\}_{s=1}^S$

**for**  $s = 1, \dots, S$  **do**

    Construct differentiable sample function

$$\tilde{f}_s(x) = \phi_s(x)^\top \theta_s$$

    Approximate level set samples

$$\mathcal{C}_s \leftarrow \text{LEVELSETAPPROX}(\tilde{f}_s, t, \tau)$$

    Initialize robust maximizer candidates

$$\mathcal{S}_s \leftarrow \text{INITIALIZEMAXIMIZATION}(\tilde{f}_s, \mathcal{C}_s)$$

    Compute robust maximizer

$$x_s^* = \arg \max_{x \in \mathcal{S}_s} -\frac{1}{\beta} \text{LogSumExp}(-d(x, \mathcal{C}_s))$$

    Compute robustness radius

$$r_s^* = \min_{c \in \mathcal{C}_s} d(x_s^*, c)$$

**end**

---

generate a set of  $n \ll k$  superlevel set points  $\mathcal{S}_s = \{x \in \mathcal{X} : \tilde{f}_s(x) \geq t\}$  by running a few iterations of gradient ascent for  $\tilde{f}_s$  starting from the level set points  $\mathcal{C}_s$  (Fig. 2 (c)). The set of  $n$  samples are selected to be maximally diverse via a max-min distance criterion; see Algorithm 19 in Appendix.

In the final step,  $x_s^*$  is found as the maximizer of the objective 12 (i.e.  $\text{LogSumExp}(-d(x, \mathcal{C}_s))$ ) by running multi-start gradient ascent starting from the points  $\mathcal{S}_s$  and picking the best one  $x_s^*$ . This approximates the robust maximizer of  $f_s$  (Fig. 2 (d)), and the associated robustness radius is  $r_s^* = \min_{c \in \mathcal{C}_s} d(x_s^*, c)$ .

**Computational cost** Even though this algorithm requires an explicit construction of the level set and involves two separate optimization stages, we can use gradient-based methods for all parts and the sampling can be done in parallel for all of the MC samples. For one posterior sample, sampling the approximate level set scales linearly with the number of approximate level set points  $k$ , basis functions  $m$ , and the dimensionality  $d$ , with complexity  $\mathcal{O}(kmd)$ . The computation of the robust maximizer is dominated by evaluating the distances between the  $n$  maximizer candidates and the  $k$  level set points, which scales as  $\mathcal{O}(knd)$ . To accelerate this phase when  $k$  is large, we restrict the LogSumExp distance computation in each iteration to the 100 nearest points, since distant points have a negligible contribution. To maintain accuracy, the set of nearest points is re-estimated every 10 iterations. In practice the computational cost is not an issue in typical applications where the function evaluation cost dominates. For example, in a three-dimensional problem the cost for selecting the next candidate was on average 2.4 seconds, only three times the cost of standard BO acquisition (2).

### 4.3 ROBUSTNESS REGRET

We propose two robustness-based regret metrics for evaluation. The *expected robustness regret* at iteration  $i$  is

$$\mathcal{R}_i = |\rho(x^*) - E_{p(\tilde{x}^*|D_i)}[\rho(\tilde{x}_i^*)]|, \quad (10)$$

where  $\tilde{x}_i^*$  denotes the estimated maximizer of Eq. (3) at iteration  $i$ . The expectation is taken over GP sample paths, and the true objective value is evaluated for each sample path. This can be computed for any method, but requires GP approximations similar to what we use in sampling. As a more tractable alternative, we define the *inference robustness regret*, relying only on the GP posterior mean:

$$\tilde{\mathcal{R}}_i = |\rho(x^*) - \rho(\tilde{x}_i^*)|, \quad (11)$$

where  $\tilde{\rho}_i$  is the posterior mean of  $\rho$  at iteration  $i$ , and  $\tilde{x}_i^*$  is the estimate of  $x^*$  at iteration  $i$ , that is  $\tilde{x}_i^* = \arg \max_{x \in \mathcal{X}} \tilde{\rho}_i(x)$ . This surrogate regret is inexpensive to compute and enables efficient evaluation.

## 5 RELATED WORK

As explained in Introduction, our setup is close to the broad literature on robust BO, especially to worst-case robust optimization, but differs by assuming perturbation-free inputs during optimization. Ignoring this distinction for the moment, the algorithm itself can be contrasted with prior work. AdveRS-2 [Saday et al., 2025] and StableOpt [Bogunovic et al., 2018] both rely on the Upper Confidence Bound (UCB) acquisition function, which scales poorly in higher dimensions. StableOpt requires solving an inner optimization problem  $\inf_{\delta \in B_r} f(x + \delta) \geq t$  for each candidate point during acquisition optimization. AdveRS-2, in turn, discretizes the design space to identify the point that maximizes the distance to the unsatisficing sub-level set, with exponential cost. We avoid both limitations. The sampling of the maximizers  $(x^*, r^*)$  is independent of the candidate and hence we can first sample the maximizers and only then optimize the acquisition function efficiently using gradients.

Information-theoretic acquisition functions have been used in Expectation Robustness setting Fröhlich et al. [2020] and in a specific worst-case contextual variable robustness use-case Weichert et al. [2024], and have long-standing roots in standard Bayesian optimization Hernández-Lobato et al. [2014]. Our formulation is closely related to the symmetric mutual-information acquisition functions [Hernández-Lobato et al., 2014, Wang and Jegelka, 2017]. It also aligns with approaches that jointly consider information about both the maximum value and its location Hvarfner et al. [2022], which has been shown to yield more informative acquisitions. Unlike their method, however, ours does not require conditioning the surrogate model on a fantasized maximum, avoiding an additional computational step.

Satisficing BO is intrinsically related to estimation of the level set where  $f(x) = t$ . Finding the level set of the true (black-box) function is an extremely hard problem, especially in higher dimensions: Bachoc et al. [2021] showed that the sample complexity depends on the packing number of the (inflated) level set, while Gotovos et al. [2013] introduced a GP-UCB-style algorithm with probabilistic convergence guarantees under the assumption that  $f$  is drawn from a GP on a discrete domain. While we need to estimate the level set within the sampling algorithm, it is critically important to note that we do *not* need the level set of the true  $f(x)$  but only the level sets of GP samples  $f_s(x)$ . These samples are smooth, we are not constrained in sample evaluations, and we can use gradients. This makes learning the level set relatively easy for reasonably smooth targets. Moreover, we do not need perfect coverage of the level set for finding the robust maximizer, as will be empirically demonstrated in Section 6.2.

One relevant line of research relates to the nature of the inputs. Fröhlich et al. [2020] and Daulton et al. [2022] consider robustness specifically for controllable inputs, whereas Weichert et al. [2024], Toscano-Palmerin and Frazier [2022] explicitly assume there are two distinct sets of inputs: design parameters to be optimized and uncontrollable environmental parameters with respect to which the solution should remain robust. Our presentation assumes all inputs will be noisy after deployment, but we can easily extend the approach to accommodate this setting by introducing a binary mask  $m \in \{0, 1\}^d$  that specifies which coordinates will be perturbed. This changes the admissible permutation set to  $B_r(m) = \{\delta \in R^d : \|m \odot \delta\| < r \text{ and } (1 - m) \odot \delta = 0\}$  but requires no other modifications.

## 6 EXPERIMENTS

We compare MRSBO with two worst-case robust optimization algorithms and one standard maximization method, explained below, using identical GP surrogate with RBF kernel implemented using BoTorch [Balandat et al., 2020] for all methods. The proxy hyperparameters are optimized after each evaluation. Each method is evaluated over 30 independent random initializations, using identical initial designs across all baselines, and the performance is evaluated using the inference robustness regret of Eq. 11, computed with respect to the maximally robust estimate  $\tilde{x}^*$  obtained based on the surrogate model. See Appendix B for complete experimental setup. The code for reproducing all experiments is available at <https://github.com/KinnunenSamuli/MRSBO/>.

**Max-value Entropy Search (MES)** Wang and Jegelka [2017] is included as a strong baseline to show how standard optimization algorithms looking for  $\arg \max_x f(x)$  ignoring robustness of the satisfiability threshold  $t$  work in our setting. MES selects points that maximize the mutual

information about the unknown maximum value  $y^*$  using  $\alpha_{MES}(x) = H(y \mid x, D_t) - E_{p(y^* \mid D_t)} [H(y \mid x, D_t, y^*)]$ .

**StableOpt** Bogunovic et al. [2018] optimizes worst-case performance under adversarial perturbations with known maximum level  $\epsilon$  with UCB acquisition

$$\alpha_{SO}(x) = \min_{\delta \in \Delta(x)} \text{UCB}(x + \delta),$$

where  $\Delta(x) = \{x' - x : x' \in \mathcal{X}, d(x, x') < \epsilon\}$ .

**AdveRS-2** Saday et al. [2025] maximizes our objective but in an adversarial environment. They maximize the optimistic robustness radius under worst-case input perturbations, subject to the constraint  $f(x) > t$ , with acquisition function

$$\alpha_{A-RS-2}(x) = d(x, x') \quad \text{s.t.} \quad \text{UCB}(x + \delta) \geq t.$$

For both robust baselines, we follow the authors’ recommendations and use  $\beta = 2$  to control the UCB exploration level. To apply these methods in our setup where the optimization is carried out without adversarial perturbations, we simulate the perturbations by drawing them uniformly within a ball of  $\epsilon$ -radius. In absence of prior knowledge the  $\epsilon$  parameter could not be determined prior to running the algorithm in realistic settings, but we consider an idealized setup where we have access to the maximally robust solution  $r^*$  and set  $\epsilon = r^*$ . This can be considered an optimal choice, but we will also illustrate the effect  $\epsilon$  has on the methods.

## 6.1 BENCHMARKING

**Illustration** Figure 3 illustrates the task and the different ways the algorithms solve it. The target is a two-dimensional Gaussian mixture with multiple modes, showcasing a typical scenario where the global maximizer of the function (at  $[0.95, 0.95]$ ) is vastly different from the true robust maximizer at  $x^* = [0.51, 0.51]$ . MES obviously finds the global maximizer and hence has substantially worse final regret due to the small robustness radius for that solution, and StableOpt that ignores the satisficing constraint converges to the widest mode – it is indeed a robust solution, but not optimal for our objective. Both MRSBO and AdveRS-2 converge (approximately) to the robust satisficing maximizer, but MRSBO achieves it an order of magnitude faster.

To provide insight on how the methods work, we show histograms of the first 20 evaluations, accumulated over the 30 repeated runs. MRSBO spends most of the evaluations exploring the parts of the level set that are informative of the robustness radius, and interestingly often does not evaluate the function at all around the eventual solution. This is in contrast with AdveRS2 that spends most acquisitions learning the function within the superlevel set. MES naturally explores the modes to find the global optimum. StableOpt explores the perimeter of the largest mode, demonstrating somewhat similar evaluation preference as ours.

Table 1: Optimization performance (mean and standard error of  $\tilde{\mathcal{R}}$ ) measured with the budget  $T = 20d$  evaluations, with bold indicating the best method for each task. See Fig. 11 in Appendix for regret curves and exact threshold values.

| Method                | MRSBO              | MES         | ADVERS2            | STABLEOPT   |
|-----------------------|--------------------|-------------|--------------------|-------------|
| <b>low threshold</b>  |                    |             |                    |             |
| Branin                | 0.14 ± 0.04        | 0.44 ± 0.08 | <b>0.09 ± 0.02</b> | 0.19 ± 0.04 |
| GM 2D                 | <b>0.00 ± 0.00</b> | 0.03 ± 0.00 | 0.01 ± 0.00        | 0.04 ± 0.01 |
| GM 3D                 | <b>0.00 ± 0.00</b> | 0.05 ± 0.01 | 0.02 ± 0.00        | 0.04 ± 0.01 |
| RP 3D                 | <b>0.13 ± 0.03</b> | 0.32 ± 0.06 | 0.14 ± 0.03        | 0.27 ± 0.07 |
| Hart. 3D              | <b>0.00 ± 0.00</b> | 0.04 ± 0.00 | 0.01 ± 0.00        | 0.05 ± 0.01 |
| <b>high threshold</b> |                    |             |                    |             |
| Branin                | 0.06 ± 0.01        | 0.09 ± 0.02 | <b>0.05 ± 0.03</b> | 0.13 ± 0.03 |
| GM 2D                 | <b>0.00 ± 0.00</b> | 0.01 ± 0.00 | 0.00 ± 0.00        | 0.03 ± 0.01 |
| GM 3D                 | <b>0.00 ± 0.00</b> | 0.03 ± 0.00 | 0.00 ± 0.00        | 0.04 ± 0.01 |
| RP 3D                 | 0.13 ± 0.03        | 0.26 ± 0.04 | <b>0.11 ± 0.03</b> | 0.22 ± 0.04 |
| Hart. 3D              | <b>0.00 ± 0.00</b> | 0.01 ± 0.00 | 0.01 ± 0.00        | 0.02 ± 0.00 |

**Low-dimensional targets** Optimization performance is evaluated on four synthetic benchmarks (Hartmann, Branin, and two Gaussian mixtures) and one real-world task (Robot Pushing Kaelbling and Lozano-Pérez [2017], Wang and Jegelka [2017]). For each function we consider three satisficing thresholds  $t$  and the optimization is initialized with 5 random evaluations.

Table 1 shows the regrets for two choices of the threshold  $t$  for each data, with evaluation budget  $T = 20d$ ; see Appendix C for full regret curves and a third (intermediate) threshold. MRSBO is in general the best solution, reaching zero regret in six scenarios, but AdveRS2 is slightly better in Branin, as well as Robot Pushing for the higher threshold. The quality of MES depends purely on how good the global maximizer happens to be; it achieves good regret when it coincides with the robust maximizer.

**Higher-dimensional targets** Previous robust BO methods are restricted to targets of very low  $d$  due to their inherent max-min optimization structure, often constructing a grid for finding the maximum of  $\alpha(x)$  and scaling exponentially with  $d$ . Our acquisition function is differentiable and we can find the maximum with gradient-based methods as in Wilson et al. [2018], and hence we avoid this scaling. Constructing the level set for sampling the robust maxima for individual samples intuitively gets harder in higher dimensions, but there is no clear computational bottleneck.

Figure 4 shows regret curves for a six-dimensional target function (Hartmann 6D) for two thresholds  $t$  for MRSBO and MES; StableOpt and AdveRS-2 would no longer scale to this. For sufficiently low threshold MRSBO retains strong performance, converging fast to a good solution. When  $t$  approaches the maximum, here  $y^* = 3.3$ , the superlevel set shrinks and all satisficing solutions are here around

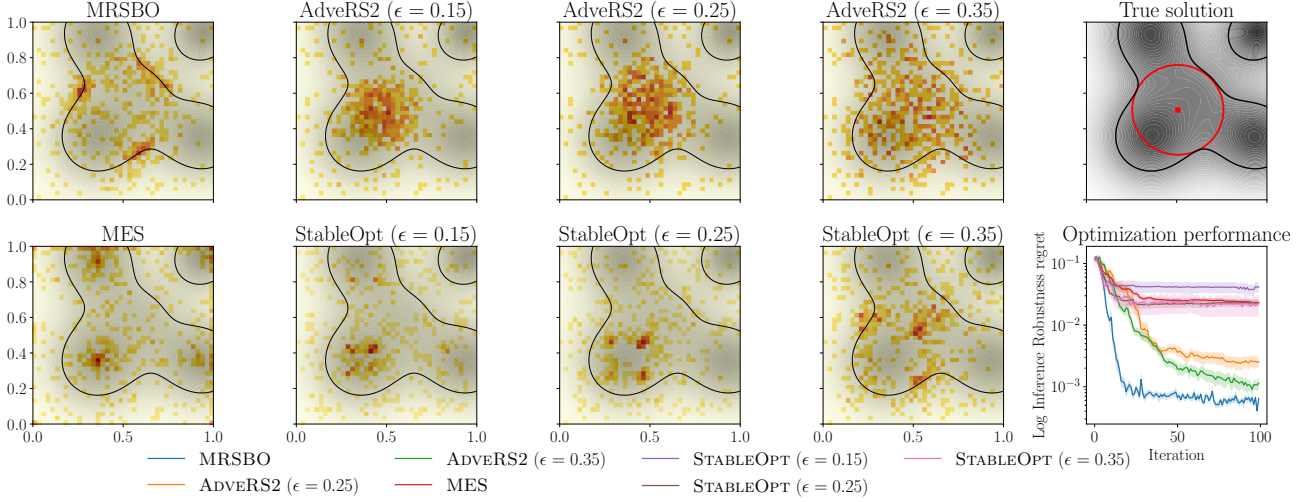


Figure 3: The first four columns show histograms of the first 20 evaluations aggregated over 30 runs for each acquisition strategy, with the right column showing the solution and regrets. MRSBO directly focuses on evaluations informative of the robustness radius and MES explores the modes to find the global optimum. StableOpt and AdveRS2 correctly explore roughly the area where the robust maximizer is, but AdveRS2 is highly inefficient and StableOpt that ignores the satisfying threshold converges to a wrong solution. For both methods, higher adversarial perturbation  $\epsilon$  increases exploration.

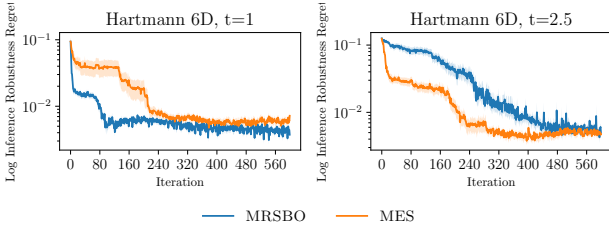


Figure 4: Performance on 6-dimensional Hartmann target.

the global maximizer. MES is then more efficient. See Appendix C for results on other problems of 4-6 dimensions.

## 6.2 LEVEL SET QUALITY

The quality of the robust maximizer and robustness radius samples  $\{x^*, r^*\}_{s=1}^S$  depends on well the level set samples  $C_s$  approximate the true level set. Since Algorithm 14 rejects samples that fail to converge, the accuracy is determined by how well  $C_s$  covers the full level set. For low-dimensional targets we can evaluate the coverage using a dense grid over the space: We identify the cells that intersect the true level set and then measure which fraction of these cells is covered, in the sense that at least one sample in  $C_s$  is in its immediate proximity, up to the grid resolution.

Figure 5 illustrates the level set coverage and its effect on the robust maximizer identification for a bi-modal Gaussian mixture of varying dimensionality, showing both the level set coverage and the fraction of runs that converge to the correct robust maximizer as a function of  $k$ , the number of level

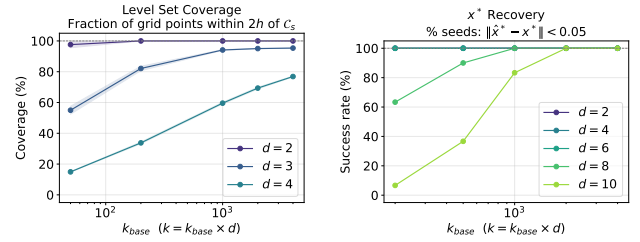


Figure 5: **Left: Level set coverage.** Fraction of the true level set of a bi-modal Gaussian mixture covered by the approximation set  $C_s$ . Since the coverage is estimated using a dense grid, the coverage is shown only for  $d \leq 4$ . **Right: Robust maximizer identification.** Fraction of robust maximizer estimates  $x_s^*$  that converge to the analytic true solution of the bi-modal Gaussian mixture.

set samples. The coverage improves with  $k$  and degrades with  $d$ , as expected. Importantly, partial coverage is often enough to find the correct maximizer. For example, with  $d = 4$  any  $k$  is sufficient for finding the correct maximizer even though the coverage is always below 80%, and even for  $d = 10$  (for which coverage cannot even be estimated) we can reliably find the true robust maximizer with large  $k$ . See Appendix A.2 for analysis of the coverage for other targets.

## 6.3 PARAMETER SENSITIVITY

Sampling of robust maximizers  $(x^*, r^*)$  depends on a few technical hyperparameters. Figure 6 shows how the robust

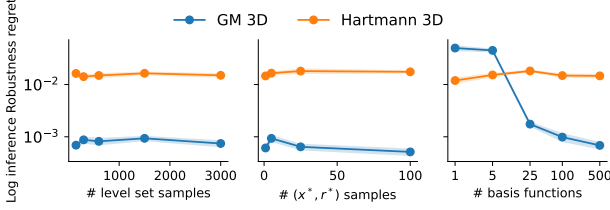


Figure 6: Sensitivity to technical choices: number of level set samples, candidates for robust maximizer, and the basis functions.

inference regret at budget  $T = 20d$  depends on these choices for two example targets. We observe that rather small values can be used, speeding up the computation: hundreds of level set samples  $k$  and tens of MC samples  $S$  are enough in practice, even though we used more in all previous experiments. However, it is important to use sufficiently many RRF basis functions  $m$ .

## 7 CONCLUSION

We introduced a new learning setup for reliable optimization of black box functions when the optimization is carried out in a reliable controlled environment but the eventual solution is deployed in an environment where the inputs may be perturbed. The goal is to find a solution that remains satisfactory, above a given threshold, under maximally large perturbations. We introduced the problem and connected it to the robust Bayesian optimization, explaining how the learning environment is different from previous works. We provided an algorithm that solves the problem using a justified information-gain acquisition, showed how it allocates the evaluations efficiently to explore the level set, and demonstrated the method outperforms standard and robust BO methods in empirical comparisons.

The key technical contribution is the algorithm for sampling robust maximizer candidates for individual GP samples, involving explicit sample-based approximation for the level set. We showed that already rather crude approximations are enough even in dimensionalities higher than typically considered in Robust BO literature, but ultimately any method relying on explicit representation of the level set will fail when the level set is highly complex, i.e. for highly non-smooth target functions in high dimensions. Despite the strong empirical performance, our analysis provides no convergence guarantees or regret bounds; establishing these is an important direction for future work. However, for such problems there are no other good solutions either. Our method is inefficient when the satisficing threshold is very close to the global function maximizer, but for this regime standard BO is a highly practical solution.

## Acknowledgements

This work was a part of Finland’s Ministry of Education and Culture’s Doctoral Education Pilot under Decision No. VN/3137/2024-OKM-6 (The Finnish Doctoral Program Network in Artificial Intelligence, AI-DOC). This work was supported by ASM International N.V. The authors acknowledge support from CSC – IT Center for Science, Finland, for computational resources. SK, PM and AK additionally acknowledge the research environment provided by ELLIS Institute Finland and support by the Research Council of Finland Flagship programme: Finnish Center for Artificial Intelligence FCAI.

## References

- Matteo Aldeghi, Florian Häse, Riley J Hickman, Isaac Tamblin, and Alán Aspuru-Guzik. Golem: an algorithm for robust experiment and process optimization. *Chemical Science*, 12(44):14792–14807, 2021.
- François Bachoc, Tommaso Cesari, and Sébastien Gerchinovitz. The sample complexity of level set approximation. In *International Conference on Artificial Intelligence and Statistics*, pages 424–432, 2021.
- Maximilian Balandat, Brian Karrer, Daniel Jiang, Samuel Daulton, Ben Letham, Andrew G Wilson, and Eytan Bakshy. BoTorch: A framework for efficient Monte-Carlo Bayesian optimization. *Advances in neural information processing systems*, 33:21524–21538, 2020.
- Felix Berkenkamp, Andreas Krause, and Angela P Schoellig. Bayesian optimization with safety constraints: safe and automatic parameter tuning in robotics. *Machine learning*, 112(10):3713–3747, 2023.
- Dimitris Bertsimas, Omid Nohadani, and Kwong Meng Teo. Robust optimization for unconstrained simulation-based problems. *Operations research*, 58(1):161–178, 2010.
- Ilija Bogunovic, Jonathan Scarlett, Stefanie Jegelka, and Volkan Cevher. Adversarially robust optimization with Gaussian processes. *Advances in neural information processing systems*, 31, 2018.
- Sathya R Chitturi, Akash Ramdas, Yue Wu, Brian Rohr, Stefano Ermon, Jennifer Dionne, Felipe H da Jornada, Mike Dunne, Christopher Tassone, Willie Neiswanger, et al. Targeted materials discovery using Bayesian algorithm execution. *npj Computational Materials*, 10(1):156, 2024.
- Hyunghun Cho, Yongjin Kim, Eunjung Lee, Daeyoung Choi, Yongjae Lee, and Wonjong Rhee. Basic enhancement strategies when using Bayesian optimization for hyperparameter tuning of deep neural networks. *IEEE Access*, 8:52588–52608, 2020.

- Samuel Daulton, Sait Cakmak, Maximilian Balandat, Michael A Osborne, Enlu Zhou, and Eytan Bakshy. Robust multi-objective Bayesian optimization under input noise. In *International Conference on Machine Learning*, pages 4831–4866. PMLR, 2022.
- Carl Doersch and Andrew Zisserman. Sim2real transfer learning for 3D human pose estimation: motion to the rescue. *Advances in Neural Information Processing Systems*, 32, 2019.
- Lukas Fröhlich, Edgar Klenske, Julia Vinogradska, Christian Daniel, and Melanie Zeilinger. Noisy-input entropy search for efficient robust Bayesian optimization. In *International Conference on Artificial Intelligence and Statistics*, pages 2262–2272. PMLR, 2020.
- Jacob Gardner, Geoff Pleiss, Kilian Q Weinberger, David Bindel, and Andrew G Wilson. GPyTorch: Blackbox matrix-matrix Gaussian process inference with GPU acceleration. *Advances in neural information processing systems*, 31, 2018.
- Alkis Gotovos, Nathalie Casati, Gregory Hitz, and Andreas Krause. Active learning for level set estimation. In *Proceedings of the Twenty-Third international joint conference on Artificial Intelligence*, pages 1344–1350, 2013.
- Daniel Hernández-Lobato, Jose Hernandez-Lobato, Amar Shah, and Ryan Adams. Predictive entropy search for multi-objective Bayesian optimization. In *International Conference on Machine Learning*, pages 1492–1501. PMLR, 2016.
- José M Hernández-Lobato, Matthew W Hoffman, and Zoubin Ghahramani. Predictive entropy search for efficient global optimization of black-box functions. *Advances in Neural Information Processing Systems*, 27, 2014.
- Carl Hvarfner, Frank Hutter, and Luigi Nardi. Joint entropy search for maximally-informed Bayesian optimization. *Advances in Neural Information Processing Systems*, 35: 11494–11506, 2022.
- Donald R Jones, Matthias Schonlau, and William J Welch. Efficient global optimization of expensive black-box functions. *Journal of Global optimization*, 13(4):455–492, 1998.
- Leslie Pack Kaelbling and Tomás Lozano-Pérez. Learning composable models of parameterized skills. In *2017 IEEE International Conference on Robotics and Automation (ICRA)*, pages 886–893. IEEE, 2017.
- Johannes Kirschner, Ilija Bogunovic, Stefanie Jegelka, and Andreas Krause. Distributionally robust Bayesian optimization. In *International Conference on Artificial Intelligence and Statistics*, pages 2174–2184. PMLR, 2020.
- Juan G Lechuz-Sierra, Ana Elvira H Martin, Ashok M Sundaram, Ruben Martinez-Cantin, and Máximo A Roa. Bayesian optimization for robust robotic grasping using a sensorized compliant hand. *IEEE Robotics and Automation Letters*, 9(11):10503–10510, 2024.
- Armin Lederer, Jonas Umlauft, and Sandra Hirche. Uniform error bounds for Gaussian process regression with application to safe control. *Advances in Neural Information Processing Systems*, 32, 2019.
- Daniel Zhuoyu Long, Melvyn Sim, and Minglong Zhou. Robust satisficing. *Operations Research*, 71(1):61–82, 2023.
- Rafael Oliveira, Lionel Ott, Vitor Guizilini, and Fabio Ramos. Bayesian optimisation for safe navigation under localisation uncertainty. In *Robotics Research: The 18th International Symposium ISRR*, pages 489–504. Springer, 2019.
- Paul Reverdy, Vaibhav Srivastava, and Naomi Ehrich Leonard. Satisficing in multi-armed bandit problems. *IEEE Transactions on Automatic Control*, 62(8):3788–3803, 2016.
- Artun Saday, Yaşar Cahit Yıldırım, and Cem Tekin. Robust satisficing Gaussian process bandits under adversarial attacks. *Advances in Neural Information Processing Systems*, 38:50381–50408, 2025.
- Alexander Shapiro, Enlu Zhou, and Yifan Lin. Bayesian distributionally robust optimization. *SIAM Journal on Optimization*, 33(2):1279–1304, 2023.
- Benjamin J Shields, Jason Stevens, Jun Li, Marvin Parasram, Farhan Damani, Jesus I Martinez Alvarado, Jacob M Janey, Ryan P Adams, and Abigail G Doyle. Bayesian reaction optimization as a tool for chemical synthesis. *Nature*, 590(7844):89–96, 2021.
- Herbert A Simon. A behavioral model of rational choice. *The quarterly journal of economics*, pages 99–118, 1955.
- Shion Takeno, Hitoshi Fukuoka, Yuhki Tsukada, Toshiyuki Koyama, Motoki Shiga, Ichiro Takeuchi, and Masayuki Karasuyama. Multi-fidelity Bayesian optimization with max-value entropy search and its parallelization. In *International Conference on Machine Learning*, pages 9334–9345. PMLR, 2020.
- Saul Toscano-Palmerin and Peter I Frazier. Bayesian optimization with expensive integrands. *SIAM Journal on Optimization*, 32(2):417–444, 2022.
- Zi Wang and Stefanie Jegelka. Max-value entropy search for efficient Bayesian optimization. In *International Conference on Machine Learning*, pages 3627–3635. PMLR, 2017.

Dorina Weichert, Alexander Kister, Sebastian Houben, Patrick Link, and Gunar Ernis. Robust entropy search for safe efficient Bayesian optimization. In *Uncertainty in Artificial Intelligence*, pages 3711–3729. PMLR, 2024.

Christopher Williams and Carl Rasmussen. Gaussian processes for regression. *Advances in Neural Information Processing Systems*, 8, 1995.

James Wilson, Frank Hutter, and Marc Deisenroth. Maximizing acquisition functions for Bayesian optimization. *Advances in neural information processing systems*, 31, 2018.

Lin Yang, Junlong Lyu, Wenlong Lyu, and Zhitang Chen. Efficient robust Bayesian optimization for arbitrary uncertain inputs. *Advances in Neural Information Processing Systems*, 36:43156–43176, 2023.

---

# Maximally Robust Satisficing Bayesian Optimization (Supplementary Material)

---

Samuli Kinnunen<sup>1</sup>

Petrus Mikkola<sup>1</sup>

Antti Niskanen<sup>2</sup>

Arto Klami<sup>1</sup>

<sup>1</sup>Department of Computer Science, University of Helsinki, Finland

<sup>2</sup>ASM International N.V.

## A METHOD DETAILS

### A.1 COMPUTATIONAL DETAILS OF THE SAMPLING FROM $p(x^*, r^* \mid D_t)$

The computational procedure of the sampling algorithm was outlined in 4.2. The method consists of two main stages. First, we construct a differentiable sample path  $f_s$  and approximate the corresponding level set. Second, we initialize robust maximizer candidates and compute the robust maximizer.

To approximate the level set (see Algorithm 14), we construct an estimate of the inflated level set

$$\mathcal{C}_s = \{x \in \mathcal{X} : |f_s(x) - t| < \tau\},$$

where  $\tau$  is a tolerance parameter. The algorithm begins by initializing candidate level set points using Latin Hypercube Sampling (LHS), which ensures space-filling coverage and promotes a uniform exploration of the domain.

For each initialized point, we solve a root-finding problem by minimizing the squared residual

$$(f_s(x_i) - t)^2.$$

Each point is optimized independently using the Gauss–Newton method. This choice is motivated by computational efficiency, as it only requires first-order derivatives while providing fast local convergence.

After constructing the approximate level set samples, we select a maximally diverse subset of these points. From this subset, gradient ascent is applied to generate superlevel set points  $S_s$ , which serve as initialization for the robust maximization step. This initialization procedure is described in 19.

The robust maximizer is obtained by solving

$$x_s^* = \arg \max_{x \in S_s} \left[ -\frac{1}{\beta} \log \sum_{c \in \mathcal{C}_s} \exp(-\beta d(x, c)) \right]. \quad (12)$$

In practice, the dominant computational cost arises from evaluating the distances between the  $n$  maximizer candidates and the  $k$  level set points, which scales as  $\mathcal{O}(knd)$ . When  $k$  is large, this step is accelerated by restricting the LogSumExp distance computation to the 100 nearest level set points at each iteration, since distant points contribute negligibly. To preserve accuracy, the nearest-neighbor set is recomputed every 10 iterations.

### A.2 ADDITIONAL ANALYSIS OF THE LEVEL SET COVERAGE

The coverage of the approximate level set  $\mathcal{C}_S$  determines the quality of the robust maximizer and robustness radius samples  $\{x^*, r^*\}_{s=1}^S$ . We analyze the level set coverage in Fig. 7 for all differentiable synthetic functions used in our experiments up to four dimensions as the analysis requires constructing a dense grid which is computationally infeasible in higher dimensions. The coverage improves with  $k$  (the number of level set samples) and decreases with  $d$ , as expected.

---

**Algorithm 3: LEVELSETAPPROX**

---

**Input:** Sample path  $\tilde{f}_s$ , threshold  $t$ , # of initializations  $n$ , tolerance  $\epsilon$ , damping  $\lambda$ , max iters  $n_{\text{iter}}$

**Output:** Approximate level set samples  $\mathcal{C}_s$

```
1 Initialize  $\mathcal{C}_s = \emptyset$ ;  
2 for  $k = 1, \dots, n$  do  
3   Sample initial point  $x^{(0)} \sim \text{LHS}$ ;  
4    $x \leftarrow x^{(0)}$ ;  
5   for  $j = 0, \dots, n_{\text{iter}} - 1$  do  
6      $r \leftarrow \tilde{f}_s(x) - t$ ;  
7     if  $|r| \leq \epsilon$  then  
8       Add  $x$  to  $\mathcal{C}_s$ ;  
9       break  
10    end  
11     $g \leftarrow \nabla \tilde{f}_s(x)$ ;  
12     $x \leftarrow x - \frac{r}{\|g\|^2 + \lambda} g$ ;  
13  end  
14 end
```

---

---

**Algorithm 4: INITIALIZEMAXIMIZATION**

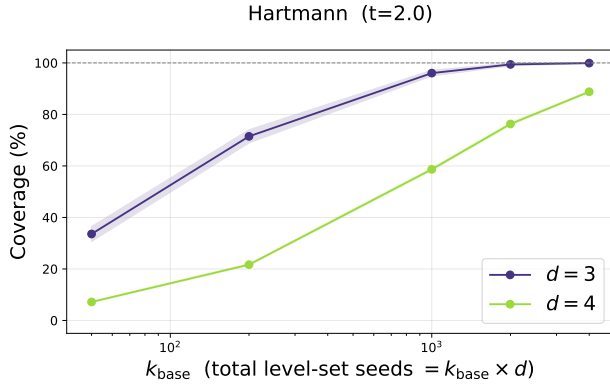
---

**Input:** Level set samples  $\mathcal{C}_s$ , sample path  $\tilde{f}_s$ , number of seeds  $n$ , learning rate  $\eta$ , ascent steps  $T$ , bounds  $(\ell, u)$

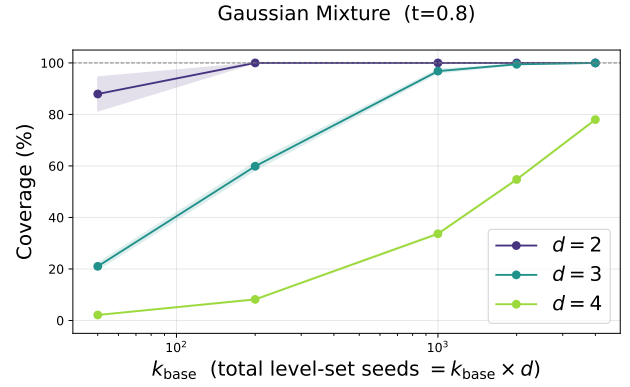
**Output:** Superlevel set points  $S_s$

```
1 Randomly select  $c^{(1)} \in \mathcal{C}_s$ ;  
2 Initialize  $\mathcal{I} \leftarrow \{c^{(1)}\}$ ;  
3 for  $k = 2, \dots, n$  do  
4   For each  $c \in \mathcal{C}_s$ , compute  $d(c) = \min_{x \in \mathcal{I}} \|c - x\|$ ;  
5    $c^* = \arg \max_{c \in \mathcal{C}_s} d(c)$ ;  
6    $\mathcal{I} \leftarrow \mathcal{I} \cup \{c^*\}$ ;  
7 end  
8  $S_s \leftarrow \emptyset$ ;  
9 for each  $x^{(0)} \in \mathcal{I}$  do  
10   $x \leftarrow x^{(0)}$ ;  
11  for  $t = 1, \dots, T$  do  
12     $x \leftarrow x + \eta \nabla \tilde{f}_s(x)$ ;  
13     $x \leftarrow \min(\max(x, \ell), u)$ ;  
14  end  
15  if  $\tilde{f}_s(x) \geq t$  then  
16     $S_s \leftarrow S_s \cup \{x\}$ ;  
17  end  
18 end  
19 return  $S_s$ ;
```

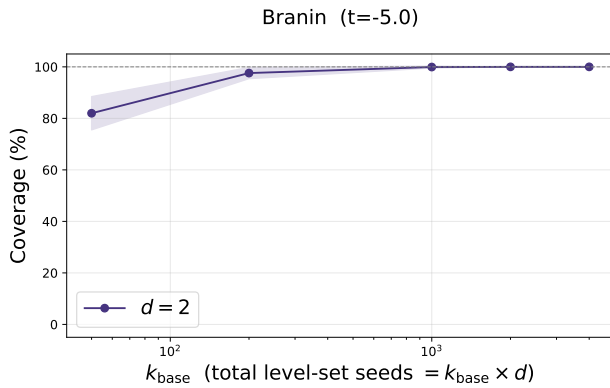
---



(a) Hartmann



(b) Gaussian Mixture



(c) Branin

Figure 7: **Level set coverage.** Fraction of the true level set, identified as cells of a dense grid that intersect the level set, covered by the approximation set  $\mathcal{C}_s$ . A cell is considered covered if at least one sample is in its immediate proximity, up to the grid resolution. Covering level sets in higher dimensions is more difficult, but increasing the sample size improves coverage monotonically.

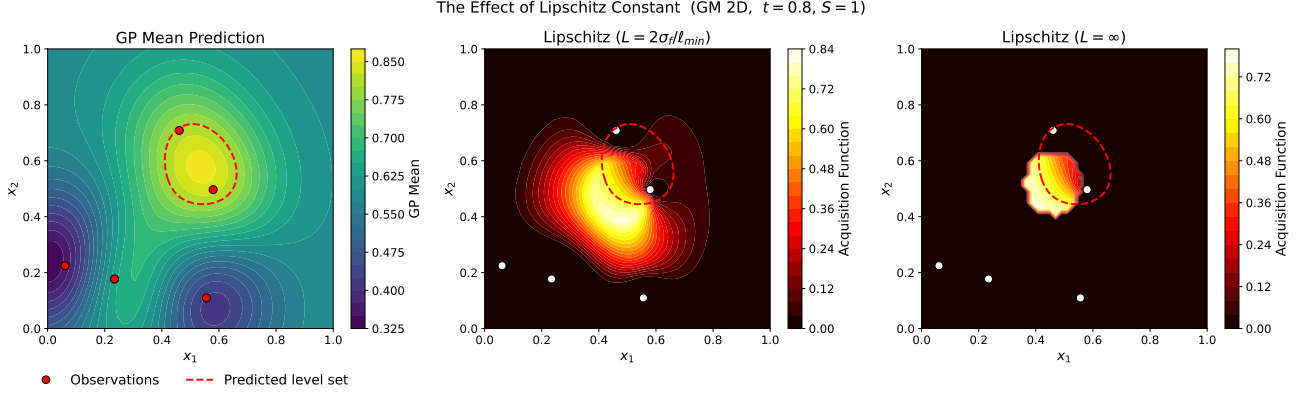


Figure 8: **Left: GP mean prediction.** The GP posterior mean fitted using 5 observations. **Middle:  $\alpha(x)$  with small  $L$ .** Acquisition function with  $L = 2\sigma_f/\ell_{min}$ , where  $\sigma_l$  is prior variance and  $\ell_{min}$  is the smallest length scale over the two dimensions. The queries expand notably outside of the robustness radius. **Right:  $\alpha(x)$  with conservative  $L = \infty$ .** Without additional smoothness assumption, the acquisitions are more concentrated and more commonly inside the robustness radius.

### A.3 CONDITIONAL ENTROPY WITH LIPSCHITZ CONTINUITY ASSUMPTIONS

Computation of the information gain in Eq. (5) requires approximating the conditional entropy

$$H(p(y \mid x, D_t, x^*, r^*)).$$

We approximate the conditional entropy by leveraging the property that if the distance to the robust maximizer satisfies  $d(x, x^*) \leq r^*$ , then by the definition of robustness, function values are constrained to  $f(x) > t$ . On the contrary, when  $d(x, x^*) > r^*$ , we do not have such bounds since the function could decrease arbitrarily fast. For sufficiently smooth functions, however, it is possible to construct a lower bound also outside the robustness radius. In the following we discuss one possible approach for doing this, pointing out challenges in obtaining practical gains from such a bound.

If we assume the function is Lipschitz continuous with Lipschitz constant  $L \geq 0$ , we can write a common lower bound that holds both within and outside of the robustness radius as

$$H(p(y \mid x, D_t, x^*, r^*)) \approx H(p(y \mid y \geq \kappa(x))) \quad (13)$$

$$= \log(\sqrt{2\pi e} \sigma(x) Z(x)) + \frac{\tilde{\kappa}(x) \varphi(\tilde{\kappa}(x))}{2 Z(x)}, \quad (14)$$

where  $\varphi$  is the probability density function of the standard normal distribution. Here  $Z(x) = 1 - \Phi(\tilde{\kappa}(x))$  can be computed using the standard normal cumulative distribution function  $\Phi$ . Now the truncation level  $\kappa(x)$  depends on  $L$ :

$$\kappa(x) = \begin{cases} t, & \text{if } d(x, x^*) \leq r^*, \\ t - L(d(x, x^*) - r^*), & \text{if } d(x, x^*) > r^*, \end{cases}$$

used here in a standardized form  $\tilde{\kappa}(x) = \frac{\kappa(x) - \mu(x)}{\sigma(x)}$ . Figure 8 shows how the acquisition function would change when using Eq. 14 for computing the information gain; it encourages the algorithm to sample candidates further outside the current robustness radius. Assuming known  $L$ , however, is far from realistic and in practice we would need to estimate it.

It is possible, but not easy, to estimate the Lipschitz constant  $L$  for a Gaussian process proxy [Lederer et al., 2019, Berkenkamp et al., 2023]. Since we only observe  $f$  through a finite set of noisy samples, we do not observe  $L$  directly but must infer it. Moreover, since  $f \sim \mathcal{GP}(\mu_t, k_t(x, x'))$  is a random function, the constant  $L$  is itself a random quantity: different sample paths of the GP admit different rates of change. Consequently,  $L$  can only be estimated up to a probabilistic

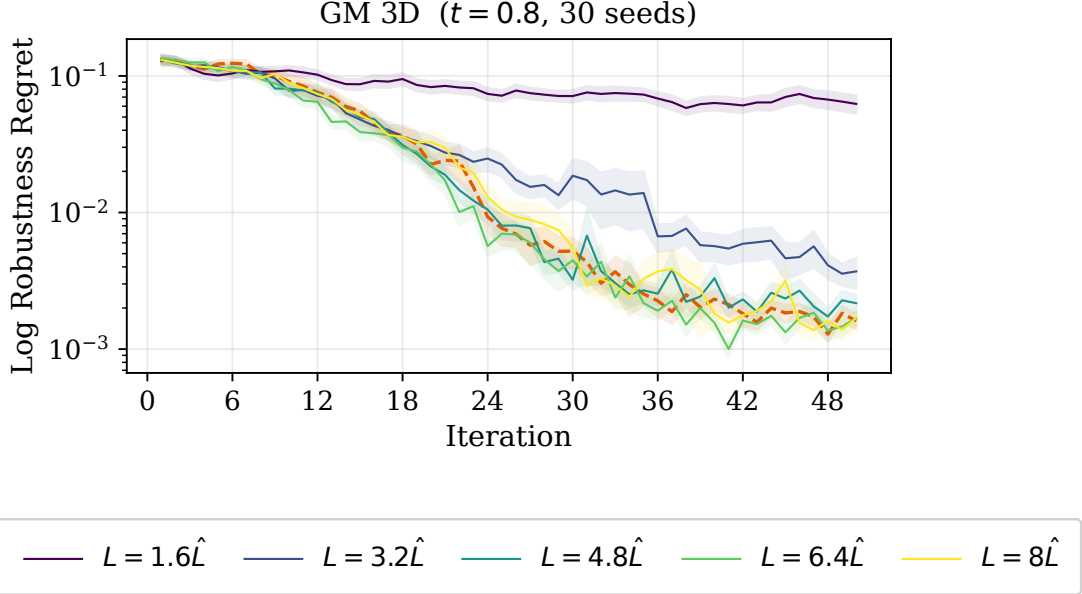


Figure 9: **Optimization performance with different Lipschitz constant estimates.** Typical estimators set  $L = c\hat{L}$  for  $\hat{L} = \frac{\sigma}{\ell_{\min}}$ , where  $\ell_{\min}$  is the smallest length scale across input dimensions and  $c$  depends on several details of the setup. We intentionally consider also overly optimistic estimators with too small  $c$ , to show the method will then fail. The most pessimistic estimator  $L = \infty$  used in all other experiments is highly competitive, but with careful finite estimate (here  $c = 6.4$ ) we may gain marginal improvement in early stages of the optimization.

bound that holds with probability  $1 - \delta$ . Lederer et al. [2019] provides a closed-form estimate  $L_f$  of a GP sample  $f$

$$L_f = \begin{bmatrix} \sqrt{2 \log \left( \frac{2d}{\delta} \right)} \max_{x \in X} \sqrt{k^{\partial_1}(x, x)} + 12\sqrt{6d} \max \left\{ \max_{x \in X} \sqrt{k^{\partial_1}(x, x)}, \sqrt{r L_k^{\partial_1}} \right\} \\ \vdots \\ \sqrt{2 \log \left( \frac{2d}{\delta} \right)} \max_{x \in X} \sqrt{k^{\partial_d}(x, x)} + 12\sqrt{6d} \max \left\{ \max_{x \in X} \sqrt{k^{\partial_d}(x, x)}, \sqrt{r L_k^{\partial_d}} \right\} \end{bmatrix} \quad (15)$$

with each element bounding the rate of change along one input dimension. Here  $k^{\partial_i}(x, x') = \frac{\partial^2}{\partial x_i \partial x'_i} k(x, x')$  is partial derivative kernel,  $L_k^{\partial_i}$  its Lipschitz constant,  $r$  the diameter of the domain  $X$ ,  $d$  the input dimension, and  $\delta \in (0, 1)$  the failure probability. All terms are computable analytically for differentiable kernels, but require kernel-specific derivations and the overall estimate depends also on the optimization domain.

Instead of constructing a specific estimator, we demonstrate the effect different estimates would have for the actual optimization quality. For the RBF kernel used in our experiments, we have  $L \propto \sigma_f/l$  and hence we plot in Figure 9 the regret as a function of the constant multiplier for this base scale. We see that clearly too optimistic estimators (small  $L$ ) fail completely and for larger multipliers the convergence curves appear highly similar to  $L = \infty$  that corresponds to arbitrarily non-smooth function and the computation collapses back to the original formulation in Eq. 7. With well-chosen  $L$  we see very slight improvement over  $L = \infty$  during the first 20-25 iterations.

Given the difficulty of estimating  $L$  and the marginal gains that can be achieved, we recommend not attempting to bound the function values outside the robustness radius.

#### A.4 CONDITIONAL ENTROPY WITH NOISY OBSERVATIONS

In the noiseless-observation case in Section 4.1, we used the approximation

$$H(p(y | x, D_t, x^*, r^*)) \approx H(p(y | y \geq \kappa(x))),$$

so the conditional distribution becomes a truncated normal with an analytical entropy. In the noisy case this implication no longer holds for the observation  $y$ ; instead it only applies to the latent function value  $f$ .

Following the derivation of the truncation with noisy observations by Takeno et al. [2020], we derive  $p(y | x, D_t, f > \kappa(x))$  using Bayes' theorem:

$$p(y | x, D_t, f > \kappa(x)) = \frac{p(f > \kappa(x) | y, x, D_t) p(y | x, D_t)}{p(f > \kappa(x) | x, D_t)}.$$

Here  $p(y | x, D_t)$  is Gaussian under the GP predictive model with observation noise, and  $p(f > \kappa(x) | x, D_t)$  is the corresponding tail probability for the latent  $f$ .

Assume the observation model  $y = f + \epsilon$  with  $\epsilon \sim \mathcal{N}(0, \sigma_\epsilon^2)$ , and the GP predictive for the latent  $f$  given  $(x, D_t)$  is  $f \sim \mathcal{N}(\mu, \sigma_f^2)$ . Then the joint marginal is

$$\begin{bmatrix} y \\ f \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} \mu \\ \mu \end{bmatrix}, \begin{bmatrix} \sigma_f^2 + \sigma_\epsilon^2 & \sigma_f^2 \\ \sigma_f^2 & \sigma_f^2 \end{bmatrix}\right).$$

Therefore,

$$f | y, x, D_t \sim \mathcal{N}(u, s^2),$$

with

$$u = \mu + \frac{\sigma_f^2}{\sigma_f^2 + \sigma_\epsilon^2}(y - \mu), \quad s^2 = \sigma_f^2 - \frac{\sigma_f^4}{\sigma_f^2 + \sigma_\epsilon^2}.$$

Hence,

$$p(f > t | y, x, D_t) = \Phi\left(\frac{u - \kappa(x)}{s}\right), \quad p(f > t | x, D_t) = \Phi\left(\frac{\mu - \kappa(x)}{\sigma_f}\right),$$

and the Bayes factor is

$$Z(y) = \frac{p(f > \kappa(x) | y, x, D_t)}{p(f > \kappa(x) | x, D_t)}.$$

Finally, the conditional entropy is

$$H(p(y | x, D_t, f > \kappa(x))) = - \int p(y | x, D_t, f > \kappa(x)) \log p(y | x, D_t, f > \kappa(x)) dy.$$

Using  $p(y | x, D_t, f > \kappa(x)) = Z(y) p(y | x, D_t)$ , this becomes

$$H = - \int Z(y) p(y | x, D_t) \log(Z(y) p(y | x, D_t)) dy,$$

which can be approximated numerically, e.g. by Monte Carlo with samples  $y_i \sim p(y | x, D_t, f > \kappa(x))$ :

$$H \approx -\frac{1}{N} \sum_{i=1}^N \log p(y_i | x, D_t, f > \kappa(x)).$$

## A.5 TECHNICAL HYPERPARAMETERS

Sampling the robust maximizers  $\{(x_s^*, r_s^*)\}_{s=1}^S$  is the most computationally demanding component of our method. Three main technical hyperparameters must be specified. First, the number of Monte Carlo samples  $S$  used to approximate the Information Gain in Eq. 5. Second, to obtain differentiable sample paths, the Gaussian process (GP) surrogate is approximated using Random Fourier Features. The number of basis functions  $m$  controls the accuracy of this approximation. Third, for each sampled path, we approximate the corresponding level set. The number of the level set samples  $k$  affects the accuracy of the estimated robustness radius: denser samples yield more precise estimates.

A sensitivity analysis of these hyperparameters is presented in Fig. 10. In this analysis, we vary one hyperparameter at a time while fixing the others to  $S = 100d$ ,  $m = 100$ ,  $k = 100$ . Overall, MRSBO is relatively robust to the choice of hyperparameters. The main exception is the number of basis functions. In the Gaussian Mixture (3D) problem, using too few basis functions leads to substantially higher regret.

Table 2: Average per-iteration runtime for selecting the next candidate in the Gaussian Mixture 3D benchmark.

| Method    | Mean [s] | Std [s] |
|-----------|----------|---------|
| ADVERS2   | 2.4      | 0.4     |
| MRSBO     | 2.4      | 0.4     |
| MES       | 0.8      | 0.1     |
| STABLEOPT | 7.8      | 1.5     |

## A.6 RUN TIME

In Table 2, we show average per-iteration runtime for selecting the next candidate in the Gaussian Mixture 3D benchmark. Only the computational cost of acquisition optimization (i.e., selecting the next evaluation point) is included. The time required for GP hyperparameter fitting is excluded, as it is identical across all methods. The runtimes are computed using Intel Core Ultra 5 125U  $\times$  14 with 16 GiB of RAM memory. The technical hyperparameters are the same as in the experiments specified in Table 4.

## B EXPERIMENT DETAILS

### B.1 IMPLEMENTATION

**Surrogate Model** We use the same GP surrogate model for all methods:

$$f(x) \sim \mathcal{GP}(\mu(x), k(x, x')),$$

where the prior mean is constant,  $\mu(x) = m$ , and the radial basis function (RBF) kernel is defined as

$$k(x, x') = \sigma_f^2 \exp \left( -\frac{1}{2\ell^2} \sum_{d=1}^D (x_d - x'_d)^2 \right),$$

where  $\sigma_f^2 > 0$  is the output scale, determining the overall magnitude of the covariance, and  $\ell > 0$  is the length-scale parameter, controlling the smoothness of the function with respect to the inputs. The term  $\sum_{d=1}^D (x_d - x'_d)^2$  denotes the squared Euclidean distance between  $x$  and  $x'$ .

The GP hyperparameters include the kernel parameters  $m$ ,  $\ell$ ,  $\sigma_f$ , and the observation noise standard deviation  $\sigma$ . After each evaluation, these hyperparameters are fitted by maximizing the log marginal likelihood:

$$\log p(\mathbf{y} | \mathbf{X}) = \log \int p(\mathbf{y} | f(\mathbf{X})) p(f(\mathbf{X}) | \mathbf{X}) df.$$

For the length-scale and output-scale parameter, we impose a Gamma prior,

$$\ell \sim \text{Gamma}(\alpha, \lambda),$$

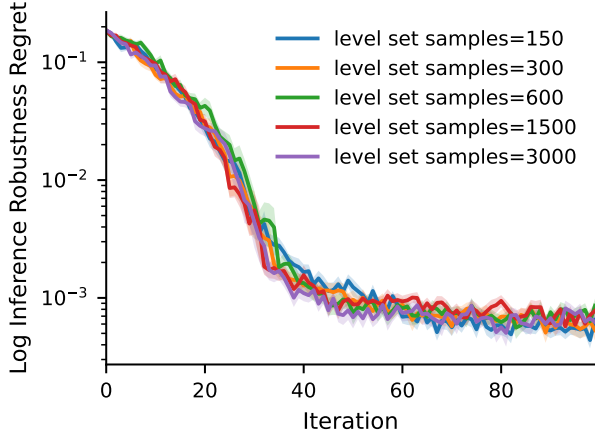
with shape parameter  $\alpha$  and rate parameter  $\lambda$  as defined in Table 3. The GP model is implemented using *SingleTaskGP* from BoTorch Balandat et al. [2020] and GPyTorch Gardner et al. [2018].

**Optimization of the Acquisition Function** For MES and MRSBO, we optimize the acquisition function using multi-start gradient descent Wilson et al. [2018] using the implementation of BoTorch.

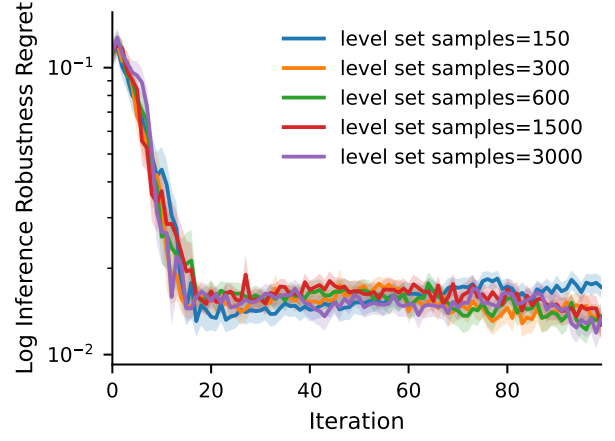
For AdveRS-2 and StableOpt, due to the discretization-based implementation of AdveRS-2 and the nested max–min optimization in StableOpt, we evaluate the acquisition function on an equidistant grid of size  $50^d$  in 2D and  $30^d$  in 3D, selecting the input that maximizes the acquisition value.

These algorithms were originally developed for adversarial optimization settings, where the maximizer of the acquisition function,

$$x_t = \arg \max_x \alpha(x),$$

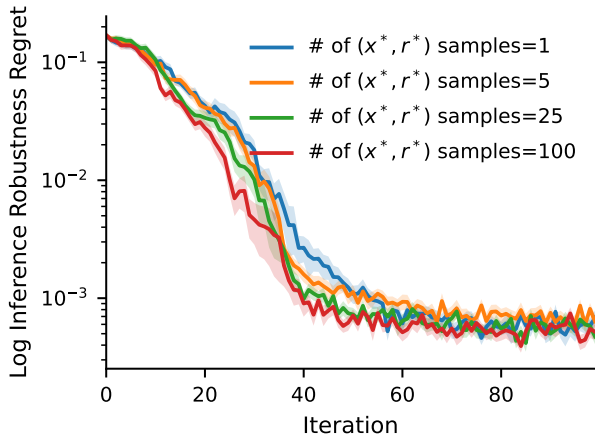


(a) GM 3D  $t = 0.8$

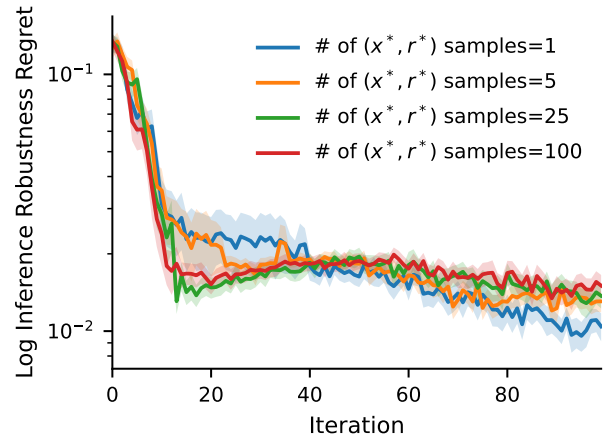


(b) Hartmann 3D  $t = 2$

(a) Sensitivity to the number of approximate level set samples

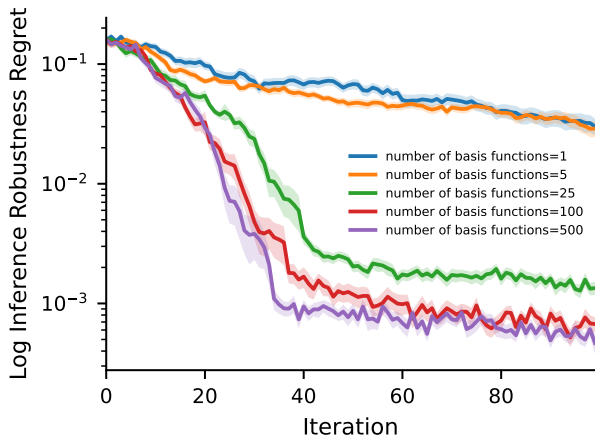


(c) GM 3D  $t = 0.8$

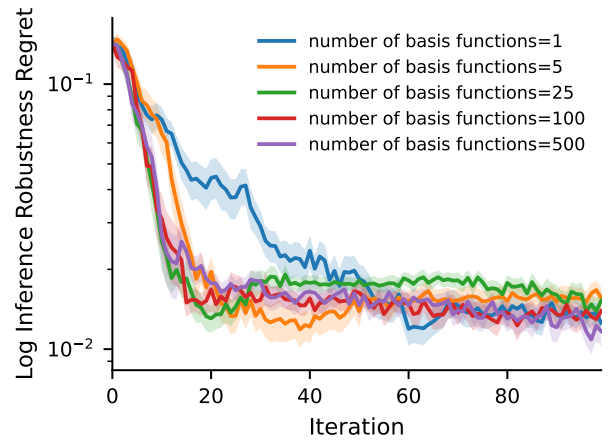


(d) Hartmann 3D  $t = 2$

(b) Sensitivity to the number of robust maximizer samples  $(r^*, x^*)$



(e) GM 3D  $t = 0.8$



(f) Hartmann 3D  $t = 2$

(c) Sensitivity to the number of basis functions

Figure 10: Sensitivity analysis across technical hyperparameters for Gaussian Mixture (GM) 3D (left column) and Hartmann 3D (right column).

Table 3: Gamma priors for the length-scale  $\ell$  and output-scale  $\sigma_f^2$  for different benchmark functions. The Gamma distribution is parameterized by concentration (shape)  $\alpha$  and rate  $\lambda$ .

| Function      | Length-scale $\ell$ |           | Output-scale $\sigma_f^2$ |           |
|---------------|---------------------|-----------|---------------------------|-----------|
|               | $\alpha$            | $\lambda$ | $\alpha$                  | $\lambda$ |
| Branin        | 1.0                 | 4.0       | 3.0                       | 2.0       |
| Hartmann 3D   | 6.0                 | 17.0      | 11.0                      | 10.0      |
| Hartmann 4D   | 6.0                 | 17.0      | 11.0                      | 10.0      |
| Hartmann 6D   | 6.0                 | 17.0      | 11.0                      | 10.0      |
| GM 3D         | 6.0                 | 17.0      | 11.0                      | 10.0      |
| GM 4D         | 6.0                 | 17.0      | 11.0                      | 10.0      |
| GM 6D         | 6.0                 | 17.0      | 11.0                      | 10.0      |
| Robot Push 3d | 3.0                 | 19.0      | 3.0                       | 2.0       |

is adversarially perturbed before evaluation:

$$y = f(x_t + \delta_t) + \epsilon.$$

In StableOpt, the perturbation is defined as the worst-case perturbation computed using the GP lower confidence bound  $\text{LCB} = \mu(x) - \beta\sigma$ :

$$\delta_t = \arg \min_{\delta \in \Delta_\epsilon(x_t)} \text{LCB}(x_t + \delta).$$

AdveRS-2 considers several perturbation schemes. In our experiments, following the experimental setup of Saday et al. [2025], we use random attack perturbations defined as

$$\delta_t \sim \text{Uniform}(\Delta_\epsilon(x_t)).$$

The perturbation  $\delta_\epsilon \in \Delta(x)$  is controlled by a stability parameter  $\epsilon$ . In our experiments, we set  $\epsilon$  equal to the admissible perturbation of the maximally robust solution, i.e.,  $\epsilon = r^*$ . However, in real-world settings, determining an appropriate value for  $\epsilon$  is challenging, since it requires prior knowledge of the input perturbation levels, which is typically unavailable.

**Technical hyperparameters.** As shown in Section A.5, the proposed method is relatively insensitive to the choice of technical hyperparameters. Therefore, we selected the values primarily according to our computational budget as shown in Tables 4 and 5.

Table 4: Technical hyperparameters for problems with dimension  $d \leq 3$ .

| Hyperparameter                               | Value |
|----------------------------------------------|-------|
| Number of Approximate level set base samples | 500   |
| Number of basis functions                    | 100   |
| Number of $(x^*, r^*)$ samples               | 100   |

Table 5: Technical hyperparameters for problems with dimension  $d \in \{4, 6\}$ .

| Hyperparameter                               | Value |
|----------------------------------------------|-------|
| Number of Approximate level set base samples | 1000  |
| Number of basis functions                    | 100   |
| Number of $(x^*, r^*)$ samples               | 100   |

## B.2 ROBOT PUSHING BENCHMARK

The robot pushing optimization problem Kaelbling and Lozano-Pérez [2017] takes a pushing action as an input and outputs the object’s distance to the target location. The objective is to minimize the distance to the target location. However, the objective is converted to a maximization problem that takes values  $f \in [0, 5]$ . The input parameters consist of:

$$r_x \in [-5, 5]$$

$$r_y \in [-5, 5]$$

$$r_\theta \in [0, 30],$$

where  $(r_x, r_y)$  denotes the robot location and  $r_\theta$  denotes pushing angle.

We use three different Satisficing thresholds  $t \in (3, 3.5, 4)$ .

We adapt the open source implementation by Wang and Jegelka [2017] available at [https://github.com/zi-w/Max-value-Entropy-Search/tree/master/test\\_functions](https://github.com/zi-w/Max-value-Entropy-Search/tree/master/test_functions).

### B.3 HARTMANN TEST FUNCTION

The  $d$ -dimensional Hartmann function (with  $d \in \{3, 4, 6\}$ ) is defined on the hypercube  $[0, 1]^d$ . We negate the function to be used in maximization defined as

$$H(\mathbf{x}) = \sum_{i=1}^4 \alpha_i \exp \left( - \sum_{j=1}^d A_{ij} (x_j - 10^{-4} P_{ij})^2 \right), \quad \mathbf{x} = (x_1, \dots, x_d)^\top \in [0, 1]^d, \quad (16)$$

where  $\alpha_i \in R$ ,  $A_{ij} \in R$ , and  $P_{ij} \in R$  are fixed coefficients.

For the six-dimensional case ( $d = 6$ ),

$$\begin{aligned} \boldsymbol{\alpha} &= (1.0, 1.2, 3.0, 3.2), \\ A &= \begin{pmatrix} 10 & 3 & 17 & 3.5 & 1.7 & 8 \\ 0.05 & 10 & 17 & 0.1 & 8 & 14 \\ 3 & 3.5 & 1.7 & 10 & 17 & 8 \\ 17 & 8 & 0.05 & 10 & 0.1 & 14 \end{pmatrix}, \\ P &= \begin{pmatrix} 1312 & 1696 & 5569 & 124 & 8283 & 5886 \\ 2329 & 4135 & 8307 & 3736 & 1004 & 9991 \\ 2348 & 1451 & 3522 & 2883 & 3047 & 6650 \\ 4047 & 8828 & 8732 & 5743 & 1091 & 381 \end{pmatrix}. \end{aligned}$$

The function is multi-modal. In the six-dimensional setting, it possesses six local minima and a global maximum at

$$\mathbf{z}^* = (0.20169, 0.150011, 0.476874, 0.275332, 0.311652, 0.6573),$$

with optimal value

$$H(\mathbf{z}^*) = 3.32237.$$

We use the open source implementation of Botorch Balandat et al. [2020]. Details about the adaptation to the lower dimensional problems available at [https://botorch.readthedocs.io/en/latest/\\_modules/botorch/test\\_functions/synthetic.html#Hartmann](https://botorch.readthedocs.io/en/latest/_modules/botorch/test_functions/synthetic.html#Hartmann)

We use three different Satisficing thresholds  $t \in (1, 2, 3)$  if  $d \leq 4$  else  $t \in (1, 2, 2.5)$ .

### B.4 BRANIN TEST FUNCTION.

The Branin function is a two-dimensional test function defined on the domain

$$(x_1, x_2) \in [-5, 10] \times [0, 15].$$

It is given by

$$B(x_1, x_2) = (x_2 - bx_1^2 + cx_1 - r)^2 + 10(1 - t) \cos(x_1) + 10, \quad (17)$$

where the constants are

$$b = \frac{5.1}{4\pi^2}, \quad c = \frac{5}{\pi}, \quad r = 6, \quad t = \frac{1}{8\pi}. \quad (18)$$

The function is multimodal and possesses three global minimizers,

$$\mathbf{z}_1 = (-\pi, 12.275), \quad \mathbf{z}_2 = (\pi, 2.275), \quad \mathbf{z}_3 = (9.42478, 2.475), \quad (19)$$

at which the global minimum value is

$$B(\mathbf{z}_i) = 0.397887, \quad i = 1, 2, 3. \quad (20)$$

We use three different Satisficing thresholds  $t \in (-20, -10, -5)$ .

We use the open source implementation of Botorch Balandat et al. [2020] available at [https://botorch.readthedocs.io/en/latest/\\_modules/botorch/test\\_functions/synthetic.html#Branin](https://botorch.readthedocs.io/en/latest/_modules/botorch/test_functions/synthetic.html#Branin).

## B.5 GAUSSIAN MIXTURE TEST FUNCTION.

The Gaussian mixture test function is defined on the unit hypercube

$$\mathbf{x} \in [0, 1]^d,$$

with  $d \in N$ . It consists of a mixture of Gaussian components, a single mode for each corner and additional mode in the center. Modes are defined as:

$$f(\mathbf{x}) = \sum_{i=1}^M a_i \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_i)^\top \boldsymbol{\Sigma}_i^{-1}(\mathbf{x} - \boldsymbol{\mu}_i)\right), \quad (21)$$

where  $M$  is the number of modes,  $a_i > 0$  are amplitudes,  $\boldsymbol{\mu}_i \in R^d$  are the component means, and  $\boldsymbol{\Sigma}_i \in R^{d \times d}$  are diagonal covariance matrices.

**The mean vectors of the Gaussians** are given by the  $2^d$  shifted corner points and one shifted center point. Let  $\mathbf{v}^{(k)} \in \{0.2, 0.8\}^d$  for  $k = 1, \dots, 2^d$  enumerate all corner vectors, and let  $\mathbf{1} \in R^d$  denote a vector of ones. Define the corner means

$$\boldsymbol{\mu}_k = \mathbf{v}^{(k)} + 0.15 \mathbf{1}, \quad k = 1, \dots, 2^d, \quad (22)$$

and define the center mean

$$\boldsymbol{\mu}_{2^d+1} = 0.5 \mathbf{1} + 0.15 \mathbf{1}. \quad (23)$$

Thus, the total number of modes is  $M = 2^d + 1$ .

**Base variances.** All modes are initialized with  $\sigma_i^2 = 0.02$  and then modified for selected modes: the first corner mode is set to  $\sigma_1^2 = 1.2 \cdot 0.02$ , the corner mode with the highest amplitude uses  $\sigma_m^2 = 0.5 \cdot 0.02 = 0.01$ , and the center mode uses  $\sigma_c^2 = 0.01$ .

**Dimension-dependent scaling.** The covariance matrices are isotropic and dimension-scaled as

$$\boldsymbol{\Sigma}_i = \sigma_i^2 d^\alpha \mathbf{I}_d, \quad (24)$$

where  $\mathbf{I}_d$  is the  $d \times d$  identity matrix and

$$\alpha = \begin{cases} 1.0, & d = 2, \\ 0.8, & d = 3, \\ 0.85, & d = 4, \\ 0.6, & d = 6. \end{cases}$$

**Amplitudes.** All amplitudes are  $a_i = 1$  except for the corner mode with the highest amplitude, where  $a_m = 1.1$ , and the center mode, where  $a_c = 0.4$ .

We use three different Satisficing thresholds  $t \in (0.7, 0.8, 0.9)$  if  $d \leq 4$  else  $t \in (1.0, 1.1, 1.2)$ .

## C ADDITIONAL RESULTS

The complete experimental results are shown in Fig. 11 and Fig. 12. For each benchmark and method, we report Robust Inference Regret curves over the full optimization horizon. These results complement the findings presented in Section 6 by providing a more comprehensive view of the empirical performance across all settings.

In addition, we include a development view of the optimization process in Figure 13, illustrating how the proposed acquisition criterion encourages exploration during the early stages and gradually shifts toward exploitation by reducing uncertainty in the relevant regions of the level set.

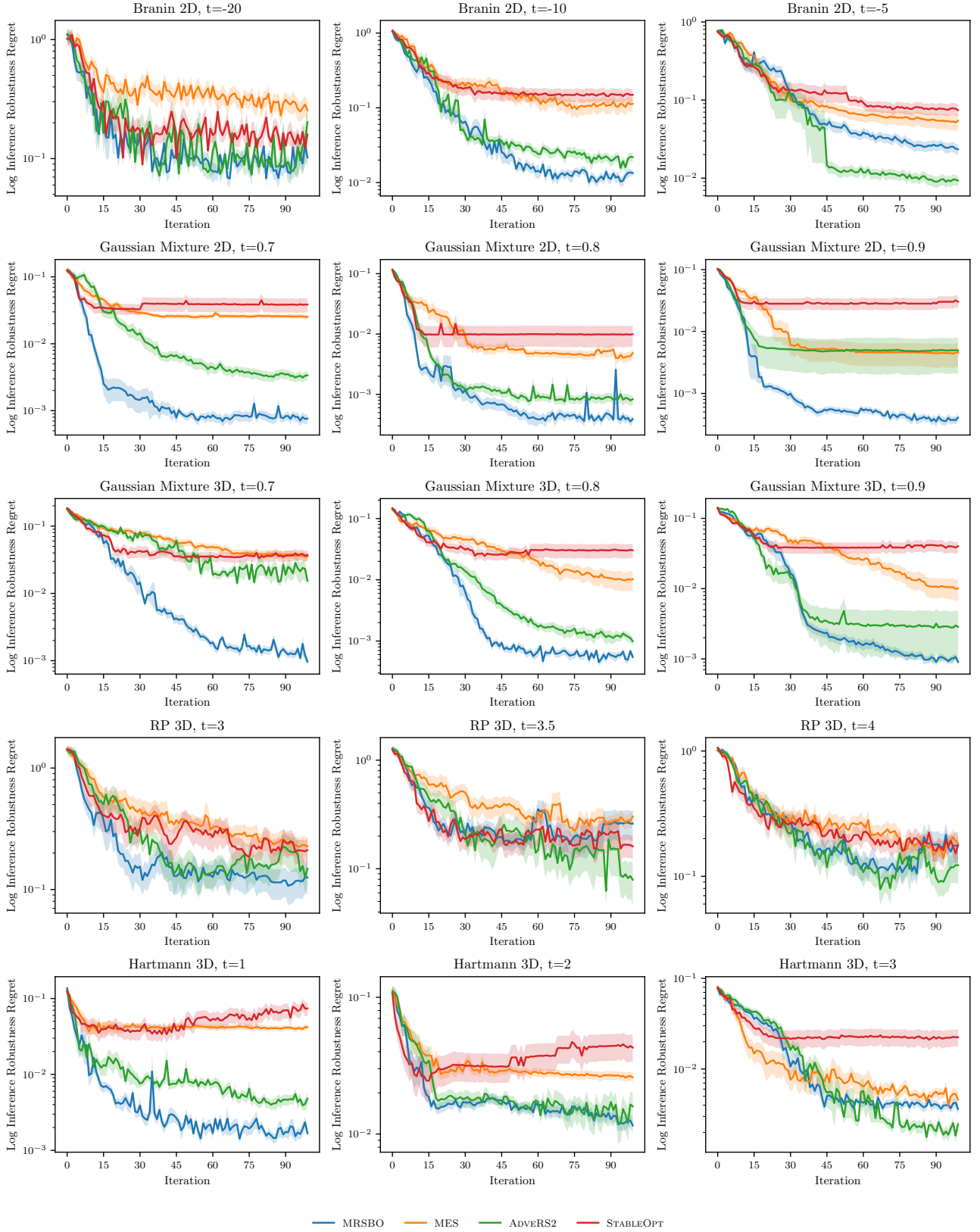


Figure 11: Comparison of AFs  $d \leq 3$

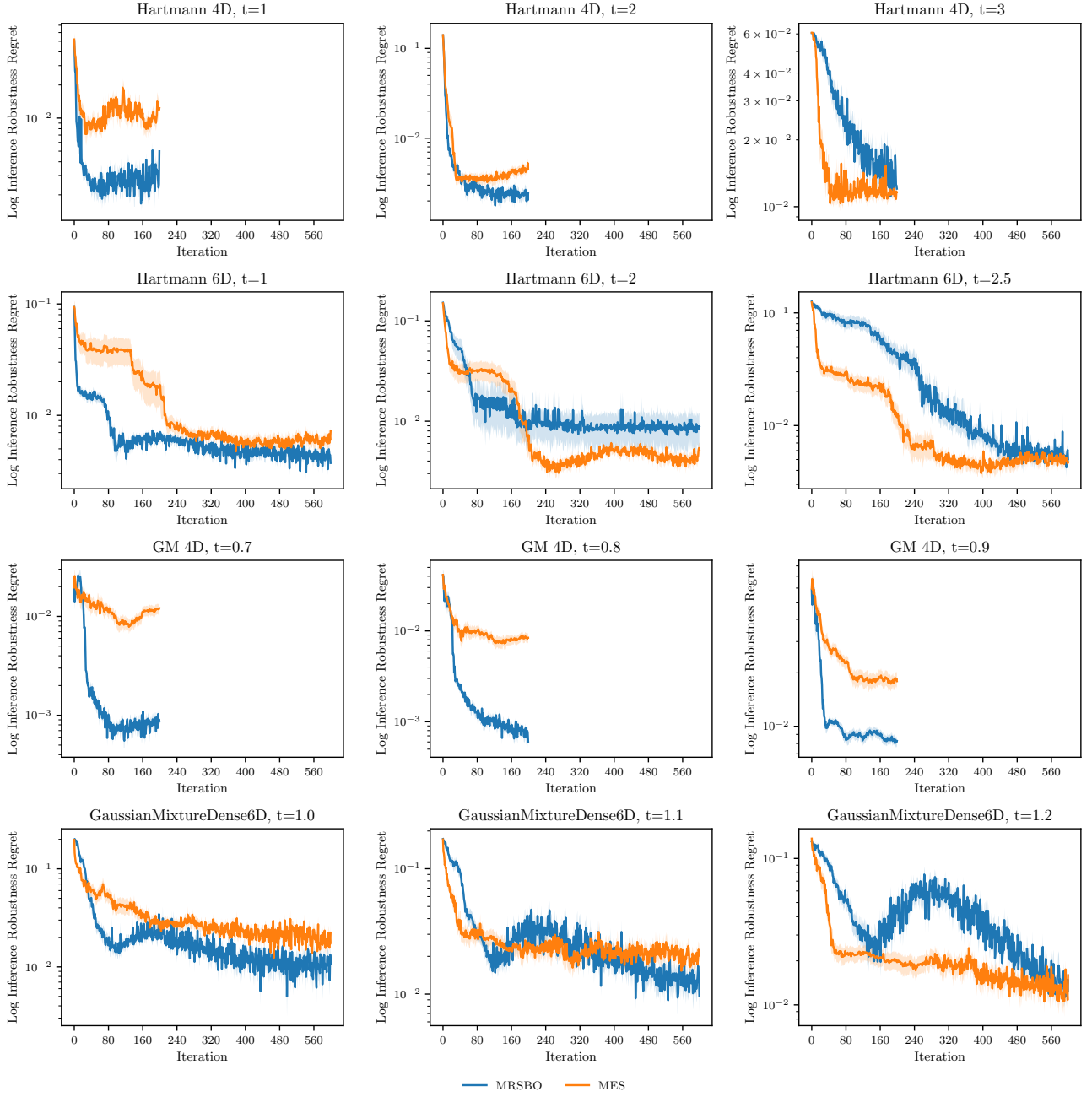


Figure 12: Comparison of AFs 4D and 6D.

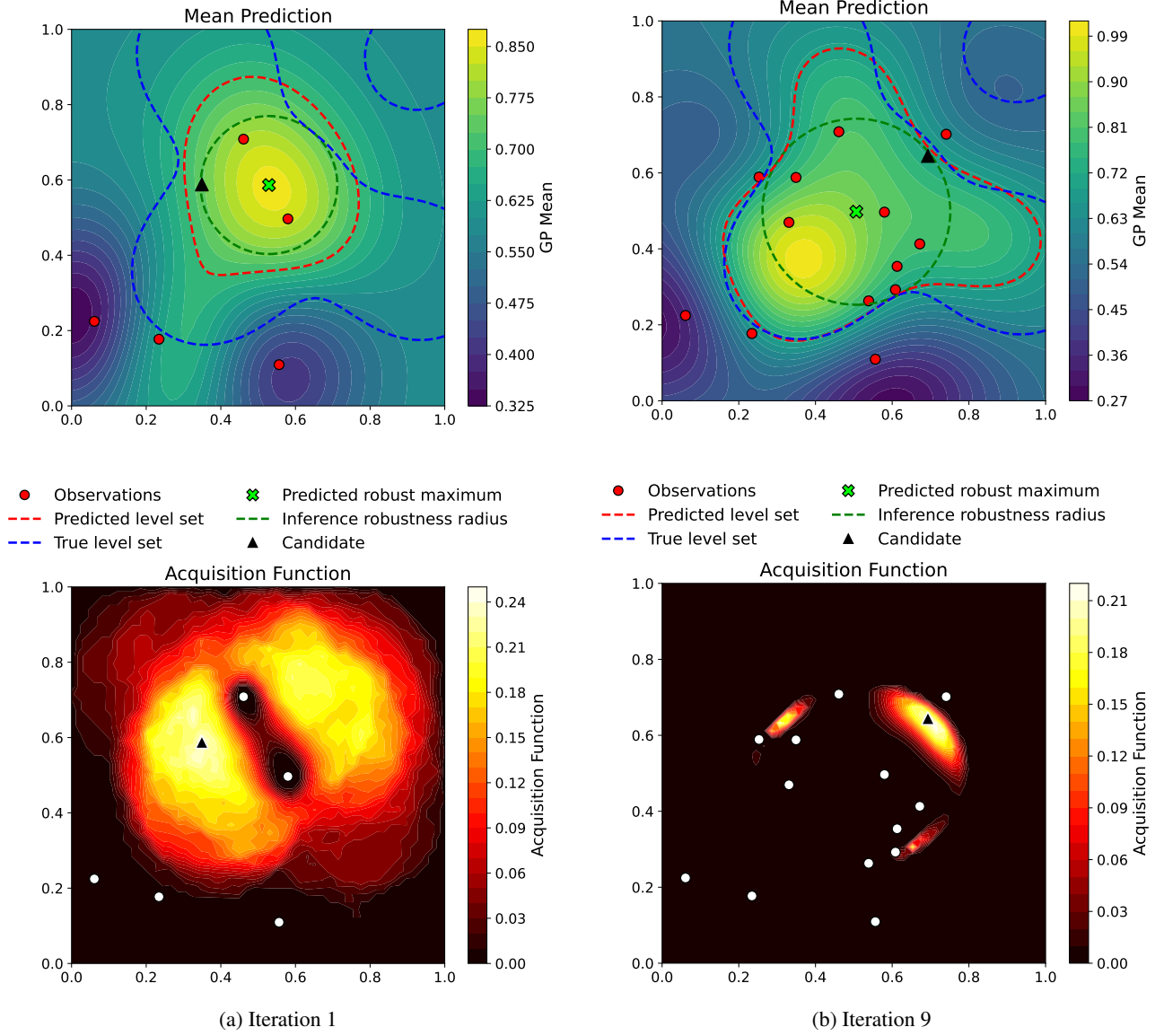


Figure 13: Progress of MRSBO algorithm in 2-dimensional Gaussian Mixture with  $t = 0.7$  and random initialization of 5 evaluations. The top panel shows the GP mean with predicted level set (red dashed), true level (blue dashed), inference robust maximizer (green marker), the selected candidate (black triangle), and observations (red points). The bottom panel shows the acquisition function with the selected candidate (black triangle). The model increasingly refines the level set approximation and concentrates exploration near the borders of the estimated maximally robust satisfying region.